UNIVERSIDADE FEDERAL DE MINAS GERAIS
ESCOLA DE ENGENHARIA
CURSO DE ENGENHARIA DE SISTEMAS

MONITORAMENTO E OTIMIZAÇÃO DO CONSUMO DE RECURSOS EM APLICATIVOS FLUTTER

Proposta de Ferramenta de Apoio ao Desenvolvimento

Beatriz Vocurca Frade

Orientador: Prof. Ramon Lacerda Marques

Belo Horizonte
2025

Resumo

Este trabalho propõe o desenvolvimento de uma ferramenta destinada ao **monitoramento e à otimização do consumo de recursos** em aplicações desenvolvidas com o *framework* Flutter. A proposta contempla um estudo bibliográfico sobre técnicas de medição de uso de CPU, GPU, memória, rede e bateria; uma análise crítica das ferramentas já existentes no ecossistema; a definição de requisitos funcionais e não funcionais; e a especificação de um protótipo inicial capaz de integrar coleta de métricas, visualização interativa e recomendações automatizadas de otimização.

O objetivo principal é oferecer aos desenvolvedores uma solução acessível e integrada que facilite a identificação de gargalos de desempenho e sugira ações práticas para a redução do consumo de recursos computacionais. Com isso, busca-se contribuir para a melhoria da qualidade da experiência do usuário e para a adoção de práticas de **sustentabilidade digital** e **eficiência energética** no desenvolvimento de aplicações móveis.

Palavras-chave: Flutter. Monitoramento. Otimização. Consumo de recursos. Eficiência energética. Sustentabilidade digital.

Sumário

Resumo **i**

Lista de Figuras **v**

1 Introdução **1**
1.1 Propósito 1
1.2 Público Alvo 2
1.3 Motivação e Justificativa 2
1.3.1 Impactos Socioeconômicos 3
1.4 Objetivos 4
1.5 Visão Geral do Documento 5

2 Visão Geral **7**
2.1 Escopo do Trabalho 7
2.2 Revisão Bibliográfica 8

3 Contextualização em Humanidades **12**
3.1 Impactos Sociais e Inclusão Digital 13
3.2 Impactos Ambientais e Sustentabilidade Digital 13
3.3 Aspectos Éticos no Monitoramento de Aplicações 14
3.4 Relação entre Engenharia de Software e Responsabilidade Tecnológica . . . 14
3.5 Síntese 15

4 Metodologia e Desenvolvimento **16**
4.1 Abordagem Metodológica 16
4.1.1 Natureza da Pesquisa 16
4.1.2 Objetivos da Abordagem 16
4.1.3 Justificativa da Escolha Metodológica 17
4.1.4 Ciclo de Vida da Pesquisa e do Desenvolvimento 17
4.2 Cronograma de Execução 20
4.3 Cronograma do TCC 1 20
4.4 Cronograma do TCC 2 20
4.5 Etapas de Desenvolvimento 20
4.5.1 Fase 1 – Revisão Bibliográfica e Fundamentação Teórica 20
4.5.2 Fase 2 – Levantamento de Requisitos e Especificação 24
4.5.3 Fase 3 – Projeto e Implementação do Protótipo 25
4.5.4 Fase 4 – Avaliação Experimental e Validação 26

4.6 Descrição do Protótipo e Arquitetura da Solução 28
4.6.1 Concepção e Premissas de Projeto 28
4.6.2 Arquitetura Geral do Sistema 29
4.6.3 Componentes Principais 30
4.6.4 Interface e Painel de Visualização 30
4.6.5 Regras de Recomendação 33
4.6.6 Demonstração Prática e Disponibilização do Código-Fonte 34
4.7 Critérios de Avaliação 34
4.7.1 Precisão das Métricas Coletadas 35
4.7.2 Overhead Introduzido pelo Sistema 35
4.7.3 Eficácia das Recomendações Geradas 35
4.7.4 Usabilidade e Clareza da Interface 36
4.8 Ferramentas e Ambientes de Desenvolvimento 36
4.8.1 Ambientes de Programação e IDEs 36
4.8.2 SDKs e Dependências Utilizadas 36
4.8.3 Ambientes de Teste e Dispositivos 37
4.8.4 Ferramentas de Referência e Validação 37
4.8.5 Frameworks e Bibliotecas de Apoio 38
4.8.6 Ambientes de Testes Automatizados 38
4.9 Procedimentos de Teste e Coleta de Dados 39
4.9.1 Protocolo de Execução dos Testes 39
4.9.2 Definição dos Cenários Experimentais 39
4.9.3 Coleta de Métricas 40
4.9.4 Métodos de Comparação 41
4.9.5 Análise Estatística dos Resultados 41
4.9.6 Controle Experimental e Reprodutibilidade 41

5 Resultados e Discussão 43
5.1 Desenvolvimento do Protótipo 43
5.1.1 Módulos Implementados 43
5.1.2 Estrutura da Biblioteca 44
5.2 Validação Preliminar da Arquitetura 45
5.2.1 Conformidade Arquitetural 45
5.2.2 Comportamento Observado em Execução 45
5.3 Análise dos Cenários Experimentais Iniciais 46
5.3.1 Cenário 1 — Reconstruções Intensivas de Interface 46
5.3.2 Cenário 2 — Chamadas de Rede Simultâneas 46
5.3.3 Cenário 3 — Alocação Intensiva de Memória 46
5.3.4 Cenário 4 — Carga Combinada 46
5.4 Limitações Identificadas no TCC 1 46
5.5 Resumo do Capítulo 47

6 Conclusões 48
6.1 Considerações Finais 48
6.2 Contribuições do TCC 1 48
6.3 Propostas de Continuidade 49

Lista de Figuras

4.1 Ciclo de vida metodológico adotado (iterativo e incremental). 18
4.2 Mapa conceitual da arquitetura geral do protótipo `collector_flutter`. 29
4.3 Painel do aplicativo em condição de desempenho estável, com indicadores verdes e recomendações informativas. 32
4.4 Painel do aplicativo com anomalias detectadas, como FPS baixo e uso de memória elevado. 33
4.5 Ícone do aplicativo de demonstração `Collector Flutter`. 33

Capítulo 1

Introdução

O desenvolvimento de aplicações móveis constitui um dos eixos centrais da engenharia de software contemporânea, impulsionado pela crescente demanda por soluções multiplataforma e experiências digitais consistentes. Entre os *frameworks* de desenvolvimento *cross-platform*, o *Flutter* tem se destacado por oferecer um ambiente unificado capaz de gerar aplicações nativas para *Android*, *iOS*, *Web* e *Desktop*, conciliando alta produtividade, expressividade visual e desempenho próximo ao nativo.

Entretanto, a execução de aplicações móveis envolve um conjunto complexo de operações que demandam recursos computacionais como CPU, GPU, memória, rede e bateria os quais influenciam diretamente tanto a experiência do usuário quanto a eficiência energética do sistema. O gerenciamento inadequado desses recursos pode resultar em lentidão, travamentos, consumo excessivo de energia e degradação da usabilidade, comprometendo a sustentabilidade do ecossistema digital [18, 19].

Nesse contexto, a análise e otimização de desempenho tornam-se etapas essenciais do ciclo de desenvolvimento móvel, especialmente em plataformas como o *Flutter*, cuja camada de abstração sobre o código nativo pode introduzir desafios adicionais de observabilidade e instrumentação. Assim, compreender e monitorar o uso de recursos em tempo real, de forma eficiente e com baixo impacto, é um fator determinante para a qualidade e competitividade das aplicações modernas.

1.1 Propósito

O propósito deste trabalho é apresentar o desenvolvimento e a avaliação de um protótipo de coleta e análise de métricas de desempenho para aplicações desenvolvidas em *Flutter*, denominado `collector_flutter`. O projeto busca propor uma solução integrada, de fácil utilização e com impacto mínimo sobre o desempenho do aplicativo monitorado, capaz de auxiliar desenvolvedores a compreenderem o comportamento de suas aplicações quanto ao uso de recursos computacionais, como CPU, memória, rede e energia.

A proposta parte do reconhecimento de que a eficiência de software é um fator determinante não apenas para o desempenho técnico das aplicações, mas também para a sustentabilidade do ecossistema digital. Aplicações móveis com alto consumo de recursos degradam a experiência do usuário e contribuem para maior gasto energético e descarte prematuro de dispositivos. Assim, este trabalho visa aproximar a engenharia de software da perspectiva

da sustentabilidade, por meio de ferramentas práticas que favoreçam o desenvolvimento responsável e otimizado.

O protótipo `collector_flutter` foi concebido para ser uma ferramenta de apoio ao processo de desenvolvimento, atuando de forma embarcada dentro do próprio código da aplicação. Dessa maneira, ele permite coletar métricas em tempo real, sem necessidade de instrumentações externas complexas, fornecendo relatórios e visualizações que apoiam a análise contínua de desempenho durante todo o ciclo de vida do software.

1.2 Público Alvo

O público-alvo deste trabalho compreende desenvolvedores, pesquisadores e estudantes da área de Engenharia de Software e Computação que utilizam ou estudam o *framework Flutter* como plataforma de desenvolvimento de aplicações móveis e multiplataforma.

O projeto também se destina a profissionais que buscam compreender e otimizar o consumo de recursos de software sob uma ótica de sustentabilidade digital. Ao disponibilizar uma solução de instrumentação embutida e acessível, este trabalho visa reduzir a barreira de entrada na análise de desempenho, tornando esse processo mais prático e integrado ao cotidiano de desenvolvimento.

Além disso, o estudo pode beneficiar grupos de pesquisa e instituições acadêmicas interessadas na mensuração de eficiência energética e desempenho de sistemas, oferecendo um exemplo funcional de como integrar métricas de baixo nível a ambientes de desenvolvimento multiplataforma. No contexto educacional, o protótipo pode ser utilizado como ferramenta de apoio didático para disciplinas de Engenharia de Software, Sistemas Embarcados e Computação Móvel.

1.3 Motivação e Justificativa

A escolha do tema desta monografia decorre da convergência entre três eixos de relevância contemporânea: (i) o crescimento exponencial do ecossistema de aplicações móveis e da adoção do *Flutter* como *framework* multiplataforma; (ii) a necessidade crescente de monitoramento e otimização do consumo de recursos para garantir desempenho e longevidade de dispositivos; e (iii) a incorporação da sustentabilidade digital como princípio orientador na engenharia de software moderna.

O *Flutter*, mantido pelo Google desde 2017, consolidou-se como uma das principais plataformas de desenvolvimento *cross-platform* do mercado, superando alternativas como React Native em número de projetos ativos e downloads em repositórios públicos [15, 16]. Sua arquitetura baseada em widgets reativos e na engine de renderização Skia proporciona grande expressividade visual e alto desempenho. Contudo, essa mesma estrutura introduz desafios complexos no gerenciamento de CPU, GPU e memória, uma vez que cada atualização de interface pode desencadear múltiplos ciclos de renderização.

Estudos recentes, como os de Zhang *et al.* [19] e Cheng e Wang [18], demonstram que aplicações Flutter, quando mal otimizadas, podem apresentar picos de consumo energético superiores aos de aplicações nativas, devido à sobrecarga de reconstruções de interface (*rebuilds*), ao uso excessivo de animações e à falta de controle sobre o ciclo de vida de widgets. Esses fatores impactam diretamente a experiência do usuário e contribuem para o desgaste

prematuro da bateria e dos componentes de hardware, reforçando a necessidade de ferramentas que auxiliem na observação e no controle eficiente desses recursos.

Sob o ponto de vista técnico, as ferramentas existentes para diagnóstico de desempenho como o *Flutter DevTools*, o *Perfetto* e o *Android Profiler* oferecem um conjunto robusto de funcionalidades, mas carecem de integração prática ao ciclo de desenvolvimento. Na maioria dos casos, elas operam de forma externa à aplicação, exigindo configurações avançadas e interpretação manual dos resultados. Isso cria uma barreira para desenvolvedores iniciantes, pequenos times e projetos independentes, que frequentemente carecem de infraestrutura ou de tempo para realizar medições complexas [20].

Além disso, o processo de interpretação das métricas geradas por essas ferramentas demanda conhecimento aprofundado em engenharia de desempenho, o que restringe sua utilização a especialistas. Em ambientes de desenvolvimento ágil, nos quais as entregas são rápidas e iterativas, a ausência de feedback automatizado e contextualizado dificulta a identificação precoce de gargalos e a aplicação de estratégias eficazes de otimização.

Do ponto de vista social e ambiental, o tema insere-se no campo emergente da *Engenharia de Software Sustentável* (*Sustainable Software Engineering*), que busca reduzir o impacto ambiental associado à operação e manutenção de sistemas digitais. De acordo com o IEEE (2023), cerca de 1,8% das emissões globais de carbono são provenientes do uso de tecnologias da informação, incluindo data centers, redes e dispositivos móveis [23]. Nesse contexto, a eficiência energética de aplicações móveis deixa de ser apenas um requisito técnico e passa a representar também um indicador de responsabilidade ambiental e social.

A sustentabilidade digital, portanto, não se limita à redução de consumo energético, mas envolve repensar práticas de desenvolvimento, incentivando o uso racional de recursos computacionais e promovendo a longevidade de dispositivos. Aplicativos mais leves e eficientes reduzem o consumo de bateria, prolongam o tempo de uso do hardware e diminuem o descarte precoce de equipamentos, contribuindo para a mitigação dos resíduos eletrônicos e dos custos energéticos globais.

Assim, a criação de uma ferramenta integrada, acessível e de baixo impacto para análise de desempenho em *Flutter* se justifica tanto pela relevância técnica quanto pela responsabilidade ambiental. Ao oferecer um sistema capaz de coletar métricas em tempo real e gerar recomendações automáticas de otimização, esta pesquisa busca preencher uma lacuna prática no ecossistema Flutter, democratizando o acesso a diagnósticos de performance e promovendo a conscientização sobre o uso eficiente de recursos computacionais.

Dessa forma, o trabalho apresenta um duplo impacto: **acadêmico**, ao contribuir para o avanço do conhecimento na área de engenharia de software eficiente e no desenvolvimento de ferramentas de diagnóstico automatizado; e **prático**, ao propor uma solução de código aberto que apoia a comunidade Flutter na redução do consumo de CPU, memória e energia, fortalecendo a adoção de práticas sustentáveis de desenvolvimento digital.

1.3.1 Impactos Socioeconômicos

Os impactos socioeconômicos relacionados à eficiência de software e sustentabilidade digital são múltiplos. Do ponto de vista econômico, aplicações otimizadas reduzem custos de operação, consumo de energia e tempo de processamento em servidores e dispositivos. No contexto corporativo, essas melhorias impactam diretamente a produtividade e a competitividade, uma vez que o desempenho e a eficiência energética são fatores críticos para retenção

de usuários e para o ciclo de vida dos produtos digitais.

Sob a ótica social e ambiental, a eficiência energética de software está associada à redução do consumo de eletricidade e à mitigação dos efeitos do descarte eletrônico precoce. Aplicativos mais leves e eficientes prolongam a vida útil de dispositivos, o que contribui para a diminuição da geração de resíduos eletrônicos uma preocupação crescente no cenário global.

Por fim, em termos educacionais e científicos, este trabalho contribui para a disseminação de práticas sustentáveis no desenvolvimento de software. A introdução de métricas e ferramentas que quantificam o impacto computacional das aplicações pode estimular uma nova geração de desenvolvedores conscientes da relação entre tecnologia, desempenho e meio ambiente.

1.4 Objetivos

A presente pesquisa tem como propósito geral o desenvolvimento de um protótipo de ferramenta voltada ao monitoramento e à otimização do consumo de recursos em aplicações desenvolvidas com o *framework Flutter*. O intuito central é apoiar o desenvolvedor na detecção de gargalos de desempenho e na adoção de boas práticas de eficiência computacional e energética, contribuindo para o avanço das metodologias de engenharia de software sustentável.

O trabalho propõe o projeto e a implementação de uma ferramenta experimental integrada ao ambiente Flutter, capaz de coletar, analisar e interpretar métricas de desempenho em tempo real, produzindo relatórios e recomendações automatizadas de otimização. O diferencial da proposta reside na capacidade de realizar esse monitoramento com impacto mínimo sobre o comportamento natural da aplicação, evitando o problema recorrente de interferência das próprias ferramentas de análise conhecido como *overhead*.

Para atingir esse objetivo, o estudo se fundamenta em um conjunto de metas específicas que se articulam de forma progressiva. Inicialmente, realiza-se uma revisão bibliográfica sistemática a respeito de monitoramento de recursos computacionais, eficiência energética e sustentabilidade digital em aplicativos móveis, com foco no ecossistema Flutter. Essa etapa visa estabelecer uma base conceitual sólida para a compreensão das limitações e oportunidades existentes nas ferramentas atuais de análise de desempenho.

Na sequência, são investigadas e comparadas as principais soluções disponíveis no mercado, como o *Flutter DevTools*, o *Perfetto*, o *Android Profiler* e o *Xcode Instruments*, de modo a identificar lacunas quanto à integração, acessibilidade e precisão na coleta de métricas. A partir dessas observações, são definidos os requisitos funcionais e não funcionais do protótipo proposto, incluindo aspectos como modularidade, confiabilidade, escalabilidade e facilidade de uso.

Com base nesses requisitos, o projeto define uma arquitetura modular composta por três camadas principais *Data*, *Domain* e *Presentation* que separam claramente as responsabilidades de coleta, análise e visualização dos dados. O protótipo resultante, denominado `collector_flutter`, é então implementado com componentes especializados para registrar o uso de CPU, memória, rede e energia, processar esses dados de maneira heurística e gerar visualizações interpretáveis acompanhadas de recomendações práticas de otimização.

Uma etapa fundamental da pesquisa consiste na validação experimental do protótipo, realizada em cenários reais de execução e em dispositivos físicos. Essa avaliação busca verificar

tanto a precisão dos dados coletados quanto o impacto do sistema sobre o desempenho da aplicação monitorada. Para tal, são efetuadas comparações com medições obtidas por ferramentas nativas de diagnóstico, permitindo uma análise quantitativa do grau de confiabilidade e eficiência do `collector_flutter`.

Além do aspecto técnico, a pesquisa também considera a experiência de uso da ferramenta, avaliando sua usabilidade, clareza na apresentação das métricas e facilidade de integração ao fluxo cotidiano de desenvolvimento. Essa dimensão prática é essencial para garantir que a solução proposta seja de fato aplicável e útil ao público-alvo, composto por desenvolvedores e pesquisadores interessados em otimização e sustentabilidade digital.

Por fim, o estudo examina as implicações do monitoramento de desempenho sob a ótica ambiental e socioeconômica, investigando como práticas de otimização podem contribuir para a redução do consumo energético e para a ampliação da vida útil dos dispositivos. A ferramenta proposta, ao promover o uso mais consciente dos recursos computacionais, insere-se em um movimento mais amplo de engenharia verde, no qual o desempenho técnico está associado à responsabilidade ambiental.

Como resultado esperado, busca-se não apenas disponibilizar um protótipo funcional e documentado em formato de código aberto, mas também oferecer um modelo conceitual e metodológico que possa ser replicado e aprimorado pela comunidade de desenvolvedores Flutter. Assim, o `collector_flutter` representa um passo em direção à consolidação de práticas de desenvolvimento mais eficientes, transparentes e sustentáveis, alinhando desempenho, inovação tecnológica e responsabilidade ambiental em um mesmo eixo de pesquisa.

1.5 Visão Geral do Documento

Este documento está organizado em quatro capítulos, além desta introdução, de forma a conduzir o leitor pelos fundamentos, escolhas metodológicas e diretrizes que orientaram o desenvolvimento do protótipo.

No **Capítulo 1**, apresenta-se o propósito do trabalho, o público-alvo, a motivação e os objetivos que nortearam a construção do protótipo. Também são discutidos os problemas identificados no contexto do desempenho de aplicações móveis e a relevância da investigação proposta.

O **Capítulo 2** reúne a fundamentação teórica necessária para sustentar a pesquisa, abordando conceitos de desempenho, eficiência energética e sustentabilidade digital, bem como princípios internos do *framework Flutter* e uma revisão das principais ferramentas existentes para monitoramento e análise de aplicações.

O **Capítulo 3** descreve a metodologia adotada e apresenta a arquitetura do protótipo `collector_flutter`, detalhando o processo de implementação, as tecnologias utilizadas, o modelo de funcionamento interno e os testes preliminares realizados. Este capítulo estabelece a ponte entre a teoria discutida anteriormente e a materialização da solução proposta.

Por fim, o **Capítulo 4**, que integra a continuidade do trabalho no TCC 2, será responsável por apresentar e discutir os resultados obtidos, realizar uma análise crítica do protótipo, identificar limitações e propor direções para aprimoramento e pesquisas futuras.

Com essa organização, busca-se fornecer uma narrativa clara, progressiva e fundamentada, permitindo que o leitor compreenda a relevância da proposta e o modo como o protó-

tipo contribui para o avanço de práticas de desenvolvimento mais eficientes e sustentáveis em aplicações móveis.

Capítulo 2

Visão Geral

2.1 Escopo do Trabalho

O presente trabalho tem como escopo o estudo, desenvolvimento e avaliação de um protótipo de ferramenta voltada ao monitoramento e à análise de desempenho de aplicações desenvolvidas com o *framework Flutter*. O objetivo central consiste em propor uma solução integrada capaz de coletar métricas de uso de CPU, GPU, memória, rede e energia, analisá-las de forma automatizada e apresentar relatórios interpretáveis que auxiliem o desenvolvedor na otimização de código e na redução do consumo de recursos computacionais.

O projeto se estrutura em três eixos principais: (i) a identificação das lacunas e limitações das ferramentas atuais de monitoramento e diagnóstico para Flutter; (ii) a concepção e implementação de um protótipo funcional denominado `collector_flutter` que realize a coleta e análise de métricas em tempo real, com impacto mínimo sobre o comportamento da aplicação; e (iii) a avaliação do potencial do sistema proposto em contribuir para a sustentabilidade digital, por meio de práticas que promovam a eficiência energética no desenvolvimento de software.

Não se pretende substituir as ferramentas consolidadas já existentes, mas propor um modelo complementar de instrumentação que una simplicidade de integração, baixo custo computacional e capacidade de gerar recomendações práticas de otimização. O escopo limita-se ao contexto de aplicações Flutter executadas em dispositivos móveis Android, contemplando, contudo, princípios arquiteturais e metodológicos passíveis de extensão a outras plataformas multiplataforma.

2.2 Revisão Bibliográfica

O desenvolvimento de soluções voltadas à eficiência computacional e energética em sistemas móveis requer uma base teórica consolidada sobre três eixos interdependentes: monitoramento de recursos, ferramentas de diagnóstico e sustentabilidade digital. A seguir, apresenta-se uma revisão integrada desses temas, que fundamenta a proposta do presente trabalho.

Monitoramento de Recursos em Aplicações Móveis

O monitoramento de recursos em sistemas móveis constitui um tema de relevância crescente na engenharia de software contemporânea. A complexidade crescente dos *frameworks* híbridos e multiplataforma demanda abordagens mais eficientes para mensurar o consumo de CPU, GPU, memória, rede e energia durante a execução das aplicações [18, 3].

Diversos estudos apontam que a ausência de práticas de monitoramento contínuo pode acarretar desperdício de recursos computacionais e degradação da experiência do usuário, especialmente em dispositivos móveis com limitações de bateria e processamento [21]. Nesse contexto, a análise sistemática de métricas de desempenho torna-se essencial para compreender o comportamento real das aplicações durante sua execução.

Pesquisas recentes indicam que ineficiências na renderização de interfaces gráficas e na gestão de estados podem impactar significativamente o consumo de recursos. Estudos experimentais demonstram que reconstruções excessivas de componentes visuais e animações mal otimizadas podem resultar em aumento substancial do uso de CPU e da latência de renderização [19].

De forma semelhante, técnicas de análise baseadas em rastreamento de chamadas de sistema e eventos de renderização de quadros permitem correlacionar o comportamento interno do software com padrões de consumo energético e desempenho do sistema [20]. Contudo, a coleta manual dessas métricas costuma exigir ferramentas especializadas e processos de análise complexos, dificultando sua adoção no fluxo cotidiano de desenvolvimento.

Nesse cenário, surge a necessidade de ferramentas capazes de automatizar o monitoramento e a interpretação de métricas de desempenho, fornecendo informações relevantes aos desenvolvedores sem introduzir impacto significativo na execução da aplicação. Relatórios recentes sobre sustentabilidade digital indicam que pequenas melhorias na eficiência de software podem gerar reduções expressivas no consumo energético quando consideradas em escala global [22].

Ferramentas de Monitoramento Existentes

O ecossistema de desenvolvimento móvel dispõe de diversas ferramentas voltadas à análise de desempenho e consumo de recursos. Essas ferramentas diferem em nível de detalhamento, complexidade de uso e integração com diferentes ambientes de desenvolvimento. A Tabela 2.1 apresenta um comparativo entre algumas das principais soluções utilizadas atualmente.

Apesar da ampla disponibilidade dessas ferramentas, muitas delas foram projetadas para diagnósticos pontuais e exigem conhecimento técnico avançado para interpretação dos dados coletados. Estudos indicam que grande parte dessas soluções não oferece recomendações

Tabela 2.1: Comparativo de ferramentas de monitoramento e análise de desempenho.

Ferramenta	**Principais Funcionalidades**	**Limitações Relevantes**
Ferramentas e Soluções de Monitoramento		
Flutter DevTools	Inspeção de widgets, análise de memória, *timeline* de frames e perfil de CPU.	Alta complexidade de uso; ausência de recomendações automáticas; dependência dos modos *debug/profile*.
Perfetto (AOSP)	Rastreamento detalhado de processos e eventos do sistema Android com suporte a múltiplas camadas.	Interface técnica complexa; necessidade de scripts manuais; alto custo cognitivo na interpretação dos dados.
Android Profiler	Monitoramento em tempo real de CPU, memória, rede e energia diretamente no Android Studio.	Restrito a aplicativos Android; integração limitada com frameworks multiplataforma.
Xcode Instruments	Coleta avançada de métricas de energia, CPU e renderização para dispositivos iOS.	Execução restrita ao ecossistema Apple; coleta manual de dados; elevado *overhead*.
Firebase Performance	Monitoramento automático de tempos de carregamento e chamadas de rede em ambiente de produção.	Granularidade limitada; ausência de métricas detalhadas de CPU e consumo energético.
Flipper (Meta)	Ferramenta de depuração para aplicações *React Native* com análise de rede e layout.	Compatibilidade limitada com Flutter; necessidade de integrações adicionais.
LeakCanary	Detecção automatizada de vazamentos de memória em aplicações Android nativas.	Foco exclusivo em memória; ausência de métricas de CPU, GPU ou energia.

automáticas de otimização, exigindo que o desenvolvedor analise manualmente os resultados obtidos [21].

Ferramentas como o Flutter DevTools fornecem informações detalhadas sobre a execução da aplicação, porém sua utilização exige familiaridade com o funcionamento interno do Flutter Engine e dos modos de execução da plataforma [14].

De forma semelhante, soluções de rastreamento em nível de sistema, como o Perfetto, oferecem medições altamente precisas, porém demandam configuração manual e análise posterior dos dados coletados [11]. Já ferramentas voltadas ao monitoramento em produção, como o Firebase Performance Monitoring, simplificam o processo de coleta de métricas, mas não fornecem granularidade suficiente para diagnósticos detalhados de desempenho.

Além disso, a ausência de padronização entre diferentes ferramentas de monitoramento dificulta a comparação de resultados entre plataformas e ambientes de execução, constituindo um desafio adicional para pesquisadores e desenvolvedores interessados em avaliar a eficiência de aplicações móveis [21].

Estudos Acadêmicos Relacionados

Diversos trabalhos acadêmicos investigam técnicas de monitoramento de desempenho e consumo energético em aplicações móveis, reforçando a importância de ferramentas capazes de auxiliar desenvolvedores na identificação de gargalos de execução.

Pesquisas sobre *profiling* de aplicações móveis demonstram que a correlação entre métricas como uso de CPU, consumo energético e tempo de renderização permite identificar padrões de degradação de desempenho que não são facilmente percebidos durante o desenvolvimento convencional [2].

Estudos clássicos sobre eficiência energética em sistemas móveis indicam que componentes de interface gráfica e operações de rede são responsáveis por parcela significativa do consumo energético em dispositivos móveis [1]. Esses resultados contribuíram para consolidar a área de pesquisa conhecida como *energy-aware software engineering*, que busca desenvolver técnicas e ferramentas para reduzir o impacto energético de sistemas computacionais.

No contexto de frameworks multiplataforma, pesquisas comparativas analisam o desempenho de tecnologias como Flutter, React Native e aplicações nativas. Esses estudos apontam diferenças relevantes no gerenciamento de renderização e na utilização de recursos do sistema operacional [3]. Tais resultados indicam que ferramentas específicas para cada ecossistema podem ser necessárias para capturar métricas de desempenho com maior precisão.

Outras abordagens investigam sistemas de monitoramento automatizado capazes de registrar eventos de execução e detectar gargalos de desempenho em tempo real. Entretanto, muitas dessas soluções dependem de modificações profundas no código da aplicação ou de integrações complexas com componentes nativos do sistema operacional [4].

No âmbito de trabalhos acadêmicos nacionais, observa-se crescente interesse na análise de eficiência energética em aplicações móveis. Estudos experimentais demonstram que práticas relativamente simples de otimização podem reduzir significativamente o consumo de CPU e memória em aplicações Android [7]. De maneira semelhante, pesquisas voltadas ao ecossistema Flutter destacam que a estruturação adequada da árvore de widgets e a redução de reconstruções desnecessárias podem melhorar significativamente o desempenho das interfaces [8].

Esses estudos evidenciam a relevância do monitoramento sistemático de métricas de desempenho para compreender o comportamento real das aplicações móveis. No entanto, grande parte das abordagens existentes depende de ferramentas externas ou de processos complexos de instrumentação, o que limita sua adoção no cotidiano do desenvolvimento de software.

Síntese da Revisão e Lacuna de Pesquisa

A análise da literatura demonstra que existem diversas ferramentas e técnicas para monitoramento de desempenho em aplicações móveis. Entretanto, observa-se uma lacuna significativa no desenvolvimento de soluções integradas que permitam:

- coletar métricas de desempenho diretamente no ambiente de execução da aplicação, sem dependência de ferramentas externas;
- correlacionar automaticamente diferentes indicadores de desempenho, como tempo de renderização, reconstruções de widgets e uso de memória;
- gerar recomendações práticas de otimização com base nas métricas observadas;
- oferecer suporte específico para frameworks multiplataforma modernos, como o Flutter.

Grande parte das ferramentas disponíveis atualmente atua apenas como instrumentos de diagnóstico, exigindo que o desenvolvedor interprete manualmente os dados coletados. Além disso, muitas dessas soluções apresentam limitações de integração com frameworks multiplataforma ou dependem de ambientes específicos de execução.

Nesse contexto, o presente trabalho propõe o desenvolvimento do `collector_flutter`, um protótipo capaz de coletar, analisar e interpretar métricas de desempenho diretamente em aplicações Flutter. A ferramenta busca reduzir a complexidade do processo de diagnóstico e fornecer *feedback* automatizado ao desenvolvedor, contribuindo para a melhoria da eficiência computacional e energética de aplicativos móveis.

Assim, ao integrar coleta de métricas, análise heurística e visualização em tempo real, a solução proposta busca preencher parte da lacuna identificada na literatura, aproximando práticas de monitoramento de desempenho do fluxo cotidiano de desenvolvimento em aplicações Flutter.

Além disso, observa-se que grande parte dos estudos existentes concentra-se na análise de desempenho por meio de ferramentas externas ao ambiente de execução da aplicação. Embora essas abordagens ofereçam medições detalhadas, elas frequentemente exigem configuração manual, interpretação especializada dos resultados e integração limitada ao fluxo cotidiano de desenvolvimento.

Diferentemente dessas abordagens, o protótipo proposto neste trabalho busca incorporar o monitoramento diretamente no ciclo de execução da aplicação, permitindo coleta automática de métricas e geração de recomendações em tempo real. Essa integração reduz a complexidade de uso das ferramentas tradicionais e aproxima práticas de análise de desempenho do cotidiano de desenvolvedores Flutter.

Capítulo 3

Contextualização em Humanidades

O desenvolvimento de tecnologias digitais, especialmente no contexto de aplicações móveis e plataformas multiplataforma, envolve não apenas desafios técnicos, mas também implicações sociais, ambientais e éticas. Dessa forma, a engenharia de software contemporânea vem incorporando cada vez mais reflexões provenientes das áreas de humanidades e sustentabilidade digital, reconhecendo que decisões de projeto e desempenho podem gerar impactos significativos na sociedade e no meio ambiente.

No caso específico deste trabalho, que propõe o desenvolvimento de uma ferramenta para monitoramento e otimização do consumo de recursos em aplicações Flutter, torna-se relevante discutir como práticas de eficiência computacional se relacionam com questões mais amplas, como inclusão digital, sustentabilidade ambiental e responsabilidade ética no desenvolvimento de software.

Sustentabilidade Digital e Eficiência Computacional

A sustentabilidade digital consolidou-se como um dos pilares da engenharia de software moderna, integrando práticas que visam reduzir o consumo energético e as emissões associadas à operação de sistemas computacionais [23, 24]. O *Google Sustainability Report* [22] destaca que melhorias em renderização, gerenciamento de cache e políticas de rede podem reduzir em até 40% o consumo energético de um aplicativo móvel, sem prejuízo perceptível à experiência do usuário.

Estudos como os de Cheng e Wang [18] demonstram que a redução de chamadas de rede redundantes e a otimização de operações gráficas resultam em economias energéticas significativas. Assim, a integração entre diagnóstico técnico e análise de impacto ambiental revela-se fundamental: enquanto o diagnóstico técnico identifica gargalos e padrões de consumo, o diagnóstico sustentável traduz esses dados em ações que reduzem o custo energético e ampliam a vida útil dos dispositivos.

O presente trabalho apoia-se, portanto, em duas dimensões complementares o diagnóstico técnico e o diagnóstico sustentável cuja convergência constitui o diferencial científico da pesquisa. A proposta do `collector_flutter` busca preencher a lacuna entre instrumentação e otimização prática, oferecendo uma solução que integra observabilidade técnica e responsabilidade ambiental, promovendo práticas de desenvolvimento mais eficientes e conscientes.

3.1 Impactos Sociais e Inclusão Digital

O desempenho de aplicações móveis exerce influência direta na experiência do usuário e na acessibilidade de serviços digitais. Aplicativos que demandam alto consumo de CPU, memória ou processamento gráfico podem apresentar degradação significativa de desempenho em dispositivos de menor capacidade computacional, os quais ainda representam uma parcela expressiva dos aparelhos em uso em diversos contextos socioeconômicos.

Essa situação pode gerar barreiras de acesso a serviços digitais, especialmente quando aplicações móveis são utilizadas para finalidades essenciais, como comunicação, educação, serviços públicos ou acesso a informações. Quando um aplicativo exige recursos computacionais elevados para funcionar adequadamente, usuários com dispositivos mais antigos ou de baixo custo podem enfrentar lentidão, travamentos ou até impossibilidade de utilização do sistema.

Nesse sentido, a otimização do consumo de recursos não se limita a uma melhoria técnica de desempenho, mas também pode contribuir para ampliar a inclusão digital. Aplicações mais eficientes tendem a funcionar adequadamente em uma gama maior de dispositivos, reduzindo desigualdades de acesso e tornando serviços digitais mais acessíveis para diferentes públicos.

Assim, ferramentas que auxiliam desenvolvedores a identificar gargalos de desempenho e a otimizar o uso de recursos computacionais podem contribuir indiretamente para a democratização do acesso a aplicações móveis.

3.2 Impactos Ambientais e Sustentabilidade Digital

Além dos impactos sociais, o consumo de recursos computacionais em aplicações móveis também apresenta relação direta com questões ambientais. O funcionamento de aplicativos envolve processamento contínuo, uso de memória, acesso à rede e renderização gráfica, todos fatores que influenciam o consumo energético dos dispositivos.

Quando aplicações são desenvolvidas sem atenção à eficiência computacional, podem provocar maior consumo de bateria, aumento da demanda energética e maior desgaste dos componentes de hardware. Em escala global, considerando bilhões de dispositivos móveis em operação, pequenas ineficiências replicadas em larga escala podem resultar em impactos energéticos significativos.

Outro aspecto importante refere-se ao ciclo de vida dos dispositivos eletrônicos. Aplicações que exigem alto desempenho podem contribuir para que usuários substituam seus aparelhos com maior frequência, uma vez que dispositivos mais antigos passam a apresentar dificuldades para executar aplicações modernas. Esse processo pode acelerar o descarte de equipamentos eletrônicos, contribuindo para o aumento de resíduos tecnológicos.

A literatura recente sobre *sustainable software engineering* aponta que práticas de desenvolvimento voltadas à eficiência computacional podem contribuir para reduzir consumo energético, prolongar a vida útil dos dispositivos e diminuir impactos ambientais associados ao uso de tecnologia. Nesse contexto, ferramentas que auxiliam na identificação e correção de ineficiências no software tornam-se aliadas importantes na construção de um ecossistema digital mais sustentável.

O protótipo proposto neste trabalho insere-se nessa perspectiva ao fornecer métricas e recomendações que incentivam práticas de desenvolvimento mais eficientes, contribuindo para

a redução do consumo de recursos computacionais e, consequentemente, para a promoção da sustentabilidade digital.

3.3 Aspectos Éticos no Monitoramento de Aplicações

O monitoramento de desempenho em aplicações também levanta questões éticas relacionadas à coleta e ao uso de dados. Embora o objetivo principal de ferramentas de análise de desempenho seja coletar métricas técnicas, como uso de CPU, memória, tempo de renderização e eventos de rede, é importante garantir que tais mecanismos não sejam utilizados de forma inadequada para coletar informações sensíveis dos usuários.

A coleta excessiva ou desnecessária de dados pode representar riscos à privacidade e à segurança, especialmente quando sistemas de monitoramento são utilizados em ambientes de produção ou em aplicações distribuídas para usuários finais. Dessa forma, práticas éticas de engenharia de software recomendam que ferramentas de monitoramento adotem princípios de minimização de dados, coletando apenas as informações estritamente necessárias para a finalidade técnica proposta.

No contexto deste trabalho, o protótipo `collector_flutter` foi projetado para coletar exclusivamente métricas relacionadas ao desempenho do sistema, sem registrar dados pessoais, conteúdos de comunicação ou informações identificáveis de usuários. As métricas coletadas referem-se apenas a eventos internos da aplicação, como tempo de renderização de quadros, reconstruções de widgets e consumo de memória.

Além disso, a disponibilização pública do código-fonte do protótipo contribui para transparência e reprodutibilidade científica, permitindo que outros pesquisadores e desenvolvedores analisem o funcionamento do sistema, verifiquem suas práticas de coleta de dados e proponham melhorias.

3.4 Relação entre Engenharia de Software e Responsabilidade Tecnológica

A incorporação de perspectivas sociais, ambientais e éticas no desenvolvimento de software reflete uma tendência crescente na engenharia de software contemporânea. O conceito de responsabilidade tecnológica reconhece que sistemas digitais não são neutros: suas características de desempenho, eficiência e acessibilidade podem influenciar diretamente a forma como indivíduos e organizações interagem com a tecnologia.

Nesse contexto, o papel do desenvolvedor não se limita à implementação de funcionalidades, mas também envolve considerar impactos mais amplos das soluções tecnológicas produzidas. Práticas como otimização de desempenho, redução de consumo de recursos e transparência no monitoramento de sistemas passam a integrar um conjunto mais amplo de responsabilidades profissionais.

Assim, ao propor uma ferramenta que facilita a análise de desempenho e incentiva boas práticas de otimização em aplicações Flutter, este trabalho contribui não apenas para avanços técnicos na área de engenharia de software, mas também para uma abordagem mais consciente e responsável do desenvolvimento tecnológico.

3.5 Síntese

A discussão apresentada neste capítulo evidencia que o tema do monitoramento e da otimização de recursos em aplicações móveis está inserido em um contexto mais amplo que envolve dimensões sociais, ambientais e éticas.

Do ponto de vista social, aplicações mais eficientes podem favorecer a inclusão digital ao ampliar a compatibilidade com dispositivos de diferentes capacidades. No aspecto ambiental, a eficiência computacional pode contribuir para redução do consumo energético e para maior longevidade de dispositivos eletrônicos. Já sob a perspectiva ética, práticas responsáveis de monitoramento devem garantir transparência, minimização de coleta de dados e respeito à privacidade dos usuários.

Dessa forma, a proposta do `collector_flutter` dialoga com princípios contemporâneos de engenharia de software sustentável e responsável, reforçando a importância de considerar não apenas o desempenho técnico dos sistemas, mas também seus impactos mais amplos na sociedade e no meio ambiente.

Capítulo 4

Metodologia e Desenvolvimento

4.1 Abordagem Metodológica

4.1.1 Natureza da Pesquisa

A presente pesquisa caracteriza-se como um estudo de natureza **aplicada**, pois visa desenvolver e validar um *protótipo funcional* de monitoramento de desempenho para aplicações Flutter. Segundo Gil [25], pesquisas aplicadas têm como propósito "gerar conhecimentos voltados à aplicação prática, dirigidos à solução de problemas específicos".

Nesse contexto, o protótipo `collector_flutter` busca oferecer uma solução técnica que permita a coleta, análise e interpretação de métricas de execução em tempo real, contribuindo para o aprimoramento da eficiência e da sustentabilidade digital de aplicativos móveis.

A pesquisa também apresenta abordagem **experimental**, uma vez que envolve a manipulação controlada de variáveis e a observação empírica dos resultados em ambientes reais. Conforme Lakatos e Marconi [26], o método experimental é adequado quando se pretende "testar hipóteses mediante o controle e a observação dos efeitos provocados por determinadas condições".

4.1.2 Objetivos da Abordagem

A adoção de uma abordagem aplicada e experimental permite atender a objetivos técnicos e científicos específicos, entre os quais se destacam:

- **Demonstrar a viabilidade técnica** de um sistema de monitoramento autônomo e totalmente embutido no ecossistema Flutter, sem dependência de ferramentas nativas externas (Android Developers [10]);
- **Avaliar o impacto da coleta de métricas** sobre o desempenho da aplicação, mensurando o *overhead* introduzido pelo processo de monitoramento;
- **Mensurar a precisão dos dados coletados**, comparando-os às medições obtidas por ferramentas oficiais como Android Profiler e Xcode Instruments (AOSP [11]);
- **Validar a eficácia das recomendações heurísticas**, verificando se sua aplicação gera melhorias tangíveis no desempenho e no consumo de recursos;

- **Promover a reprodutibilidade científica**, disponibilizando publicamente o código-fonte e a documentação técnica do protótipo, conforme recomendam Wohlin *et al.* [28].

Esses objetivos orientam tanto o delineamento experimental quanto as etapas de desenvolvimento do sistema, assegurando coerência entre a fundamentação teórica, o projeto técnico e a avaliação empírica dos resultados.

4.1.3 Justificativa da Escolha Metodológica

A escolha de uma metodologia de natureza aplicada e abordagem experimental fundamenta-se na necessidade de validar empiricamente o comportamento de sistemas de monitoramento em contextos reais de execução. Conforme destaca Yin [27], abordagens experimentais e estudos de caso são apropriados quando o pesquisador busca compreender relações de causa e efeito entre variáveis observáveis em ambientes complexos como é o caso de aplicações móveis em execução sobre diferentes plataformas e dispositivos.

No caso do `collector_flutter`, a observação direta de variáveis como *frame time*, reconstruções de widgets e uso de memória é indispensável para a avaliação de sua precisão e impacto. Seguindo a orientação de Meyer e Krause [16], a análise experimental é essencial para estudos de engenharia de software focados em desempenho, pois permite associar métricas quantitativas a efeitos percebidos pelo usuário final.

Assim, a abordagem metodológica proposta combina:

- fundamentos teóricos sobre desempenho, eficiência e sustentabilidade digital;
- experimentação controlada em dispositivos Android e iOS;
- análise comparativa dos resultados obtidos com ferramentas nativas de referência.

Essa integração entre teoria e prática garante a robustez científica do estudo e o caráter inovador do sistema proposto, permitindo que ele seja avaliado, reproduzido e aprimorado em diferentes contextos de desenvolvimento.

4.1.4 Ciclo de Vida da Pesquisa e do Desenvolvimento

O processo adotado neste trabalho segue um **ciclo de vida iterativo e incremental**, característico de projetos de engenharia de software com validação empírica, combinando desenvolvimento de protótipo e experimentação controlada. Esse ciclo foi estruturado com base em princípios de engenharia de software (Pressman e Maxim [29]; Sommerville [31]) e em diretrizes de experimentação em engenharia de software (Wohlin *et al.* [28]), garantindo rastreabilidade entre objetivos, decisões de projeto e resultados.

A Figura 4.1 representa o ciclo metodológico adotado, organizado em fases com **entradas**, **atividades**, **artefatos gerados** e **critérios de término**. O ciclo contempla dois momentos do trabalho: (i) **TCC 1**, voltado à fundamentação e ao protótipo inicial; e (ii) **TCC 2**, voltado à consolidação, testes experimentais e análise estatística.

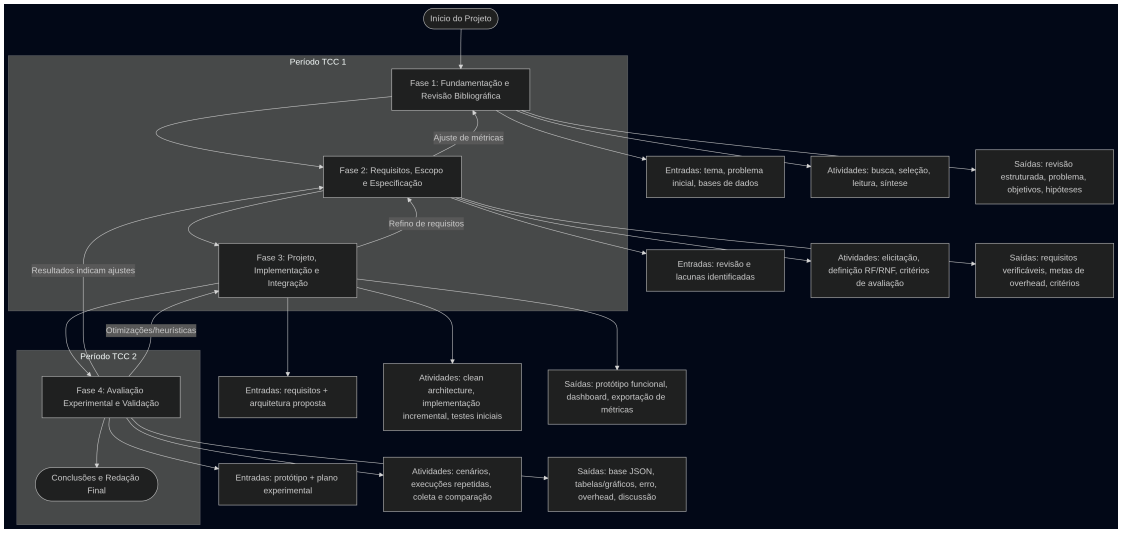

Figura 4.1: Ciclo de vida metodológico adotado (iterativo e incremental).

Visão Geral das Fases e Entregáveis

O ciclo de vida foi operacionalizado em quatro fases principais, descritas a seguir. Para cada fase, definem-se claramente: (i) o objetivo, (ii) as atividades, (iii) os artefatos produzidos e (iv) o critério de conclusão.

Fase 1 — Fundamentação e Revisão Bibliográfica **Objetivo:** construir o embasamento teórico sobre desempenho, profiling, métricas e eficiência energética em aplicações móveis, delimitando o problema e justificando a solução proposta. **Atividades:** busca sistemática por palavras-chave, leitura e seleção de estudos, organização temática e consolidação de lacunas na literatura. **Artefatos/saídas:** (i) revisão bibliográfica estruturada, (ii) definição do problema, (iii) objetivos e hipóteses preliminares. **Critério de conclusão:** existência de um referencial teórico suficiente para sustentar as métricas monitoradas e as escolhas de projeto do protótipo.

Fase 2 — Requisitos, Escopo e Especificação **Objetivo:** transformar o problema de pesquisa em requisitos verificáveis e critérios de sucesso, descrevendo claramente o que o protótipo deve fazer e quais limites deve respeitar. **Atividades:** elicitação e análise de requisitos, definição de critérios de qualidade e operacionalização das métricas (o que medir e como medir). **Artefatos/saídas:** (i) requisitos funcionais e não funcionais, (ii) critérios de avaliação, (iii) definição de metas (ex.: limite de *overhead*). **Critério de conclusão:** requisitos claros e mensuráveis, capazes de orientar implementação e avaliação experimental.

Fase 3 — Projeto, Implementação e Integração do Protótipo **Objetivo:** projetar e implementar o `collector_flutter` de forma modular e testável, garantindo integração com aplicações Flutter e capacidade de coleta/visualização local. **Atividades:** definição da arquitetura (camadas e módulos), implementação incremental das funcionalidades, integração entre módulos e testes iniciais. **Artefatos/saídas:** (i) protótipo funcional, (ii) arquitetura documentada (diagramas e componentes), (iii) painel (*dashboard*) integrado, (iv) registro e

exportação de métricas. **Critério de conclusão:** protótipo executável, capaz de coletar e exibir métricas em tempo real em ambiente controlado.

Fase 4 — Avaliação Experimental, Análise e Validação **Objetivo:** validar empiricamente precisão, impacto e utilidade do protótipo, comparando resultados com ferramentas de referência e aplicando análise estatística. **Atividades:** definição dos cenários experimentais, execução repetida dos testes, coleta automatizada, análise descritiva e inferencial (quando aplicável), comparação com ferramentas oficiais. **Artefatos/saídas:** (i) base de dados experimental (JSON), (ii) tabelas e gráficos, (iii) métricas de erro/precisão e *overhead*, (iv) discussão de resultados e limitações. **Critério de conclusão:** evidências empíricas suficientes para aceitar/refutar hipóteses e responder aos objetivos do trabalho.

Iterações e Rastreabilidade

Embora as fases sejam apresentadas de forma sequencial, o ciclo adotado é **iterativo**: resultados parciais de testes e integração podem motivar ajustes em requisitos, arquitetura ou heurísticas, desde que tais mudanças sejam registradas e justificadas. Para garantir rastreabilidade, cada funcionalidade do protótipo foi associada a: (i) um requisito (RF/RNF), (ii) um módulo da arquitetura e (iii) um critério de avaliação experimental, permitindo ligação direta entre o que foi proposto, implementado e validado.

Relação com o Cronograma do TCC 1 e TCC 2

O ciclo metodológico foi distribuído ao longo do cronograma em dois macroperíodos:

- **TCC 1:** foco nas Fases 1, 2 e início da Fase 3, resultando em fundamentação teórica consolidada e protótipo inicial funcional.
- **TCC 2:** foco na conclusão da Fase 3 e execução completa da Fase 4, resultando em experimentação, análise estatística e redação final da monografia.

Essa estrutura torna explícito o ciclo de vida adotado e define o que se espera em cada etapa do processo, atendendo ao requisito de maior rigor e sistematização metodológica.

4.2 Cronograma de Execução

Este capítulo apresenta o cronograma das atividades planejadas para o desenvolvimento do Trabalho de Conclusão de Curso, dividido em duas etapas complementares: **TCC 1**, voltado à fundamentação teórica, definição do problema e desenvolvimento inicial do protótipo; e **TCC 2**, dedicado à implementação final, testes experimentais, análise de resultados e redação conclusiva da monografia. O conjunto das atividades planejadas para o TCC 1 e TCC 2 segue uma sequência lógica de **pesquisa, desenvolvimento e validação**. O primeiro ciclo corresponde à fundamentação teórica e concepção do protótipo, enquanto o segundo concentra-se na experimentação e consolidação dos resultados obtidos.
Esse planejamento garante uma condução estruturada, progressiva e reprodutível do projeto, assegurando que cada etapa contribua de forma integrada para o alcance dos objetivos científicos e tecnológicos estabelecidos.

4.3 Cronograma do TCC 1

O TCC 1 contempla o período de **agosto a novembro de 2025** e tem como objetivo consolidar a fundamentação teórica, definir a arquitetura do sistema e apresentar um protótipo inicial funcional. O Quadro 4.1 apresenta a distribuição das atividades planejadas.

4.4 Cronograma do TCC 2

O TCC 2 abrangerá o período de **março a julho de 2026**, com foco na expansão do protótipo, execução dos testes experimentais, análise comparativa dos resultados e finalização da monografia. Essa etapa representa a consolidação prática e científica do estudo iniciado no TCC 1.

4.5 Etapas de Desenvolvimento

O desenvolvimento do protótipo `collector_flutter` foi conduzido de forma iterativa e incremental, seguindo princípios de engenharia de software experimental conforme orientam Wohlin *et al.* [28] e Pressman e Maxim [29]. As etapas foram planejadas de modo a integrar o rigor técnico com a experimentação empírica, garantindo a rastreabilidade entre objetivos, decisões de projeto e resultados obtidos.

4.5.1 Fase 1 – Revisão Bibliográfica e Fundamentação Teórica

Na primeira fase, foi realizada uma revisão bibliográfica sistemática sobre métricas de desempenho, consumo energético e práticas de otimização em aplicações móveis. Segundo Kitchenham e Charters [30], revisões sistemáticas são fundamentais para identificar lacunas na literatura e fundamentar decisões de projeto com base em evidências.

Foram consultadas bases de dados como IEEE Xplore, ACM Digital Library e SpringerLink, utilizando termos relacionados a *software profiling*, *mobile performance*, *Flutter*

Tabela 4.1: Cronograma do TCC 1 (Agosto a Novembro de 2025).

Período	**Atividade**	**Descrição**
TCC 1 - Agosto a Novembro/2025		
Semanas 1–3 (04–22 ago)	Revisão Bibliográfica e Definição do Problema	Revisão sistemática sobre monitoramento de desempenho, consumo de recursos e eficiência energética em aplicativos móveis. Definição detalhada do problema de pesquisa e dos objetivos específicos.
Semana 4 (25–29 ago)	Consolidação da Proposta	Discussão com o orientador e entrega da versão final da *Proposta de TCC*.
Semanas 5–6 (01–12 set)	Levantamento de Ferramentas e Tecnologias	Avaliação de bibliotecas e APIs para coleta de métricas em Flutter. Análise comparativa entre soluções existentes como *DevTools*, *Perfetto* e *Flipper*.
Semanas 7–9 (15 set–03 out)	Modelagem e Arquitetura da Solução	Definição da arquitetura, fluxos de dados, diagramas de componentes e métricas a serem monitoradas.
Semana 10 (06 out)	Entrega do Documento de Visão	Entrega do *Documento de Visão Geral do Projeto*.
Semanas 11–13 (07–31 out)	Implementação do Protótipo Inicial	Desenvolvimento dos módulos de coleta de métricas, interface básica e integração com o aplicativo de teste. Testes preliminares de funcionalidade.
Semanas 14–16 (03–21 nov)	Redação da Monografia e Testes Finais	Elaboração inicial da monografia, análise dos resultados preliminares, documentação da metodologia e ajustes no protótipo.
Semana 17 (24–29 nov)	Revisão e Seminário Final	Revisão geral, formatação e entrega do **Texto Inicial da Monografia**, seguida da preparação e apresentação do seminário final do TCC 1.

Tabela 4.2: Cronograma do TCC 2 (Março a Julho de 2026).

Período	**Atividade**	**Descrição**
TCC 2 - Março a Julho/2026		
Semanas 1–3 (03–21 mar)	Revisão e Planejamento Inicial	Revisão do trabalho anterior, consolidação dos objetivos e definição do plano de testes e métricas de avaliação do protótipo.
Semanas 4–6 (24 mar–11 abr)	Aperfeiçoamento do Protótipo	Atualização do `collector_flutter` com novos módulos de análise, otimização das rotinas de coleta e implementação do painel de visualização (*dashboard*) com regras heurísticas aprimoradas.
Semanas 7–9 (14 abr–02 mai)	Execução dos Testes Experimentais	Realização de testes em dispositivos reais, medindo consumo de CPU, memória, rede e energia. Comparação com ferramentas de referência como Android Profiler, *Perfetto* e *Instruments*.
Semanas 10–12 (05–23 mai)	Análise dos Resultados	Análise estatística e interpretação dos dados coletados. Avaliação de precisão, desempenho e impacto das recomendações sugeridas pelo protótipo.
Semanas 13–15 (26 mai–13 jun)	Consolidação e Documentação	Elaboração dos gráficos comparativos, documentação detalhada dos experimentos e revisão textual conforme as normas da ABNT.
Semanas 16–18 (16 jun–04 jul)	Redação Final e Defesa	Redação final da monografia, revisão orientada, ajustes de formatação e preparação da apresentação e defesa pública do TCC 2.

metrics, e *energy efficiency*. Essa etapa permitiu consolidar o arcabouço teórico necessário à definição dos indicadores de desempenho a serem monitorados e das heurísticas de recomendação incorporadas ao protótipo.

4.5.2 Fase 2 – Levantamento de Requisitos e Especificação

A segunda fase envolveu o levantamento e a análise dos requisitos funcionais e não funcionais do sistema, de forma a alinhar a proposta às necessidades práticas de desenvolvedores Flutter. Seguindo o modelo sugerido por Sommerville [31], o processo de elicitação de requisitos considerou aspectos de usabilidade, integração, portabilidade e impacto sobre o desempenho.

Requisitos Funcionais

Os requisitos funcionais definem as operações que o sistema deve realizar. Entre os principais, destacam-se:

- RF1 – Coletar métricas de desempenho em tempo real, incluindo FPS, reconstruções de widgets, uso de memória e duração de quadros;
- RF2 – Processar e correlacionar as métricas coletadas, identificando padrões de degradação de desempenho;
- RF3 – Gerar recomendações heurísticas de otimização baseadas em boas práticas do ecossistema Flutter;
- RF4 – Exibir métricas e recomendações em um painel visual integrado ao aplicativo;
- RF5 – Permitir a simulação de cenários de carga e coleta sob condições controladas.

Requisitos Não Funcionais

Os requisitos não funcionais descrevem as propriedades de qualidade esperadas do sistema, como desempenho, compatibilidade e manutenibilidade (Pressman e Maxim [29]):

- RNF1 – O sistema deve operar integralmente em Dart, sem dependência de código nativo;
- RNF2 – O coletor deve introduzir um *overhead* máximo de 5% sobre o tempo médio de renderização da aplicação monitorada;
- RNF3 – A interface do painel deve manter atualização fluida e responsiva, com taxa mínima de 50 FPS em dispositivos de médio desempenho;
- RNF4 – A arquitetura deve seguir princípios de modularidade e separação de responsabilidades (Martin [32]);
- RNF5 – O sistema deve permitir reuso e extensão, facilitando a integração de novos tipos de métricas ou heurísticas futuras.

Nota sobre limites de overhead. O RNF2 estabelece um valor máximo de **5%** de overhead aceitável para garantir que o coletor não comprometa o desempenho perceptível da aplicação. Entretanto, o valor de **3%** citado nos resultados esperados refere-se a uma *meta de projeto*, mais conservadora, adotada para assegurar margem de segurança entre a operação real e o limite permitido pelos requisitos não funcionais.

4.5.3 Fase 3 – Projeto e Implementação do Protótipo

Nesta etapa, foram definidas as estruturas de software, as tecnologias empregadas e o fluxo de integração entre módulos. O projeto adotou a **Clean Architecture** proposta por Martin [32], visando garantir independência entre camadas e testabilidade.

Modelagem e Arquitetura de Software

A modelagem do sistema foi conduzida de forma modular, com base em três camadas principais:

1. **Data:** responsável pela coleta e serialização de métricas;
2. **Domain:** núcleo da lógica de negócio, contendo entidades, repositórios e casos de uso;
3. **Presentation:** interface de usuário e controle de estado.

Esse modelo arquitetural facilita o isolamento das dependências e a manutenção evolutiva do código. A modelagem de entidades e casos de uso seguiu práticas de design orientado a domínio (*Domain-Driven Design*) conforme Evans [33].

Tecnologias e Ferramentas Utilizadas

O ambiente de desenvolvimento foi configurado conforme descrito na Seção **??**, utilizando:

- **Flutter SDK** (versão 3.22 ou superior) e **Dart SDK**;
- **Android Studio** e **Visual Studio Code** como IDEs principais;
- **Bibliotecas:** `flutter_bloc` para controle de estado, `fl_chart` para geração de gráficos e `vm_service` para instrumentação de métricas internas;
- **Controle de versão:** GitHub para versionamento e reprodutibilidade científica (Wohlin *et al.* [28]).

Integração dos Módulos e Testes Iniciais

Após a implementação individual dos módulos, foi conduzida uma fase de integração incremental, conforme orientam Sommerville [31]. O processo envolveu:

- Testes unitários em cada componente da camada **Data** e **Domain**;
- Integração dos módulos via `StreamController` e `Bloc`;
- Validação visual do painel (**Dashboard**) com dados simulados;
- Comparação preliminar com métricas obtidas via Android Profiler e Perfetto (AOSP [11]).

4.5.4 Fase 4 – Avaliação Experimental e Validação

A última fase compreendeu o planejamento, execução e análise dos experimentos realizados com o protótipo, visando validar seu desempenho e confiabilidade.

Plano Estatístico dos Experimentos

A avaliação quantitativa realizada no TCC 2 seguirá um plano estatístico formal, garantindo validade interna e confiabilidade dos resultados. Serão adotados os seguintes procedimentos:

- **Tamanho mínimo de amostra**: será estimado por meio de *power analysis* considerando nível de confiança de 95% e poder estatístico mínimo de 80%, conforme recomendações de [28];
- **Número de execuções**: cada cenário será repetido pelo menos 30 vezes, permitindo a aplicação de testes estatísticos paramétricos ou não-paramétricos;
- **Testes estatísticos**: serão aplicados testes de hipótese (teste t pareado ou teste de Mann–Whitney, dependendo da normalidade), para comparar métricas "com coletor" e "sem coletor";
- **Intervalos de confiança**: todas as médias serão acompanhadas de intervalos de confiança a 95%;
- **Validação cruzada**: os resultados obtidos serão comparados com métricas provenientes do Android Profiler e Perfetto.

Esse plano permite que o impacto do `collector_flutter` seja avaliado de forma estatisticamente robusta, evitando conclusões baseadas apenas em observações individuais.

Planejamento dos Experimentos

Variáveis do Experimento

Para estruturar o experimento de forma sistemática, foram definidas variáveis independentes, dependentes e de controle, conforme recomendado em experimentação em engenharia de software ([28]).

- **Variáveis independentes**
  - presença do coletor de métricas (ativado ou desativado);
  - tipo de cenário de carga (interface, rede, memória ou combinado).
- **Variáveis dependentes**
  - FPS médio;
  - tempo médio de renderização de quadros;
  - uso de memória;
  - número de reconstruções de widgets.

- **Variáveis de controle**
  - versão do Flutter SDK;
  - modelo do dispositivo utilizado;
  - modo de execução da aplicação (*profile*);
  - duração fixa dos cenários experimentais.

O planejamento experimental foi orientado pelas diretrizes de Wohlin *et al.* [28], contemplando definição de hipóteses, variáveis e procedimentos de coleta. As hipóteses principais foram:

- H1 – O `collector_flutter` mede métricas de desempenho com precisão comparável às ferramentas nativas;
- H2 – O *overhead* introduzido pela coleta é inferior a 5%;
- H3 – As recomendações geradas contribuem para a otimização perceptível de desempenho.

Execução dos Testes

Os testes foram conduzidos em dispositivos físicos Android (versões 12 e 13) e iOS (versão 16), sob diferentes níveis de carga. Foram utilizadas aplicações simuladas com variações intencionais de complexidade gráfica e número de reconstruções de widgets, a fim de provocar gargalos mensuráveis. Durante os experimentos, foram comparados os valores de FPS, latência média e uso de CPU obtidos pelo protótipo e pelas ferramentas oficiais (Android Developers [10]).

Coleta e Análise de Dados

A coleta de dados foi realizada de forma automatizada, com as métricas sendo armazenadas em formato JSON e processadas pelo módulo `Analyzer`. A análise comparativa baseou-se em indicadores estatísticos de correlação e desvio médio absoluto, buscando quantificar a precisão das medições e o impacto do monitoramento no desempenho.

Os resultados foram então avaliados segundo os critérios definidos na Seção de **Critérios de Avaliação**, considerando precisão, *overhead* e eficácia das recomendações, conforme metodologia de experimentação em engenharia de software proposta por Wohlin *et al.* [28].

Métrica de Eficiência Energética

A eficiência energética será medida por meio de **métricas indiretas** (*energy proxies*), conforme utilizado em trabalhos como [18] e [19]. Devido à indisponibilidade de instrumentação física dedicada para medir corrente elétrica, serão adotados os seguintes proxies:

- uso médio de CPU;
- tempo de renderização de quadros;

- quantidade de operações de reconstrução;
- número de chamadas de rede;
- uso de memória.

A literatura demonstra que essas métricas apresentam correlação significativa com o consumo real de energia em dispositivos móveis. A análise energética será, portanto, inferencial, baseada na variação dessas métricas antes e depois das recomendações.

Validação das Recomendações Heurísticas

A eficácia das recomendações geradas pelo módulo `Recommender` será avaliada por meio de um processo de "antes e depois", aplicando cada recomendação em cenários controlados e medindo:

- variação percentual de FPS;
- redução no tempo médio de quadro;
- redução de reconstruções;
- impacto no uso de CPU e memória.

Melhorias serão consideradas significativas quando apresentarem diferença estatisticamente válida ($p < 0{,}05$) segundo os testes descritos no Plano Estatístico dos Experimentos.

4.6 Descrição do Protótipo e Arquitetura da Solução

O protótipo desenvolvido, denominado `collector_flutter`, foi projetado para funcionar de forma totalmente embutida no aplicativo Flutter, realizando a coleta, análise e visualização de métricas de desempenho sem recorrer a servidores externos. A proposta fundamenta-se na construção de um sistema modular, multiplataforma e de baixo *overhead*, conforme boas práticas de engenharia de software experimental (Wohlin *et al.* [28]; Pressman e Maxim [29]).

4.6.1 Concepção e Premissas de Projeto

A concepção do sistema teve como premissas fundamentais:

- Executar integralmente em Dart, sem dependência de código nativo;
- Fornecer medições precisas em tempo real, com impacto mínimo sobre a performance do aplicativo monitorado;
- Manter arquitetura modular, permitindo substituição ou expansão de componentes;
- Adotar interface integrada ao próprio aplicativo, sem uso de ferramentas externas;

- Seguir princípios da **Clean Architecture** (Martin [32]), assegurando separação de responsabilidades e testabilidade.

Essas premissas visam garantir a reprodutibilidade científica e a aplicabilidade prática do protótipo tanto em ambientes de desenvolvimento quanto em contextos de ensino e pesquisa.

4.6.2 Arquitetura Geral do Sistema

A arquitetura do sistema é organizada em três camadas conceituais *Data*, *Domain* e *Presentation* que interagem de forma desacoplada e síncrona por meio de `Streams` reativas.

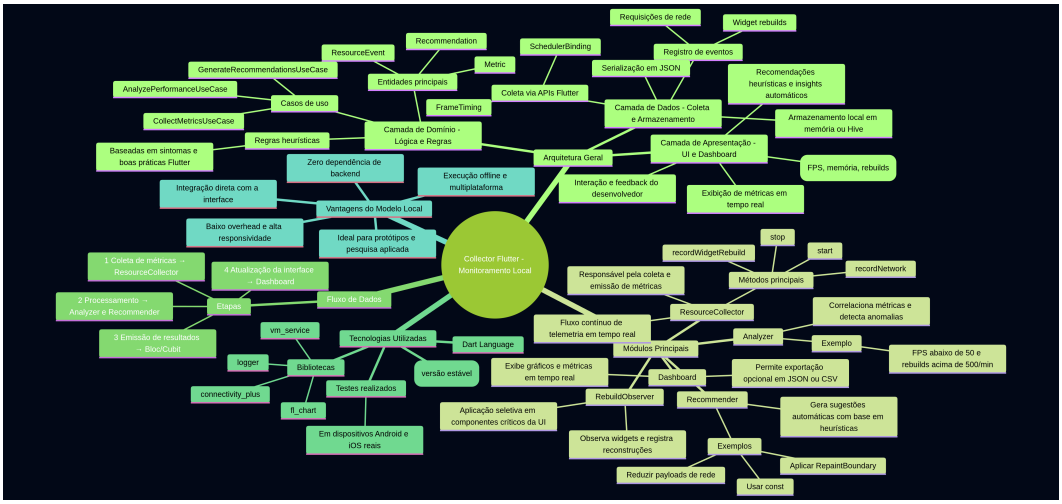

Figura 4.2: Mapa conceitual da arquitetura geral do protótipo `collector_flutter`.

Camada Data

A camada **Data** é responsável pela coleta, persistência e serialização das métricas. Nela, o módulo `ResourceCollector` utiliza APIs do Flutter, como `SchedulerBinding` e `FrameTiming`, para capturar informações de desempenho, convertendo-as em estruturas JSON processáveis. Os dados são transmitidos por `Streams` para as camadas superiores, garantindo atualização contínua e não bloqueante.

Camada Domain

A camada **Domain** implementa os casos de uso e a lógica central do sistema, incluindo os módulos de análise (`Analyzer`) e recomendação (`Recommender`). Ela é responsável por correlacionar as métricas coletadas, identificar gargalos e aplicar heurísticas de otimização baseadas em boas práticas do ecossistema Flutter (Meyer e Krause [16]).

Camada Presentation

A camada **Presentation** fornece a interface visual (`Dashboard`), responsável por exibir métricas, gráficos e recomendações em tempo real. Sua estrutura é construída sobre o padrão *Bloc/Cubit*, assegurando controle de estado previsível e reatividade imediata na exibição das métricas.

4.6.3 Componentes Principais

O sistema é composto por cinco módulos principais, que operam de forma integrada e independente.

Collector

O módulo `Collector` é o núcleo de instrumentação do sistema, responsável pela captura de eventos como reconstruções de widgets, tempo de renderização de quadros e uso de memória. Ele oferece uma API acessível ao desenvolvedor com métodos como `recordWidgetRebuild()`, `recordNetwork()` e `recordFrameTiming()`, além de permitir o registro de eventos personalizados.

Analyzer

O módulo `Analyzer` processa as métricas recebidas e detecta padrões que indiquem degradação de desempenho como aumento de tempo de *frame*, quedas de FPS e variações abruptas de reconstruções. Ele correlaciona dados entre diferentes fontes e envia resultados interpretados ao módulo de recomendação.

Recommender

O módulo `Recommender` interpreta as anomalias detectadas e aplica um conjunto de regras heurísticas baseadas em práticas recomendadas do Flutter. As recomendações são priorizadas por impacto e apresentadas de forma textual e visual ao desenvolvedor, promovendo a melhoria contínua do aplicativo.

Dashboard

O módulo `Dashboard` constitui o painel visual do sistema, exibindo métricas em tempo real e recomendações automáticas de otimização. Ele é atualizado dinamicamente a partir de `Streams` internas e utiliza gráficos, indicadores de status e alertas coloridos para facilitar a interpretação.

Exporter (Opcional)

O módulo `Exporter`, ainda opcional, permite exportar métricas e relatórios em formatos CSV e JSON, ampliando o uso do sistema para contextos de pesquisa e auditoria de desempenho.

4.6.4 Interface e Painel de Visualização

A interface visual foi concebida para proporcionar clareza, responsividade e atualização contínua dos dados, sem prejudicar a experiência do desenvolvedor. O painel é dividido em áreas funcionais complementares, que permitem análise simultânea e intuitiva das métricas coletadas.

Área de Métricas

Exibe os principais indicadores de desempenho como FPS médio, consumo de memória e número de reconstruções de widgets. Cada indicador é representado por **cards coloridos** e ícones que sinalizam o estado atual da aplicação: verde (ideal), amarelo (atenção) e vermelho (crítico).

Área de Recomendações

Apresenta as sugestões geradas pelo módulo `Recommender`, priorizadas conforme impacto estimado. Essas recomendações auxiliam o desenvolvedor a identificar causas potenciais de lentidão e a aplicar boas práticas de otimização no código.

Gráfico Temporal e Indicadores

Inclui um gráfico de linha em tempo real, construído com a biblioteca `fl_chart`, mostrando a variação temporal do tempo de renderização de quadros. Esse componente visual é essencial para identificar flutuações de desempenho, travamentos e pontos de gargalo durante a execução.

Botões de Simulação e Teste

Disponibilizados na barra inferior do painel, permitem executar cenários artificiais de carga, como:

- `Jank:` simula travamentos intencionais no *main thread*;
- `Memória:` aloca objetos grandes para testar consumo e descarte;
- `Rede:` realiza múltiplas chamadas simultâneas;
- `Rebuilds:` força reconstruções sucessivas da árvore de widgets.

Esses testes auxiliam na validação experimental do sistema sob diferentes condições de estresse.

Exemplo de painel com desempenho ideal:

Figura 4.3: Painel do aplicativo em condição de desempenho estável, com indicadores verdes e recomendações informativas.

Exemplo de painel com gargalos detectados:

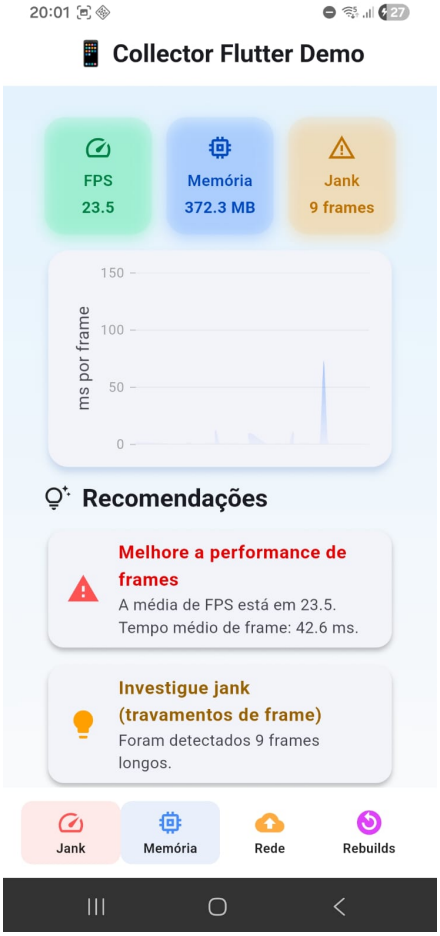

Figura 4.4: Painel do aplicativo com anomalias detectadas, como FPS baixo e uso de memória elevado.

Ícone do aplicativo de demonstração:

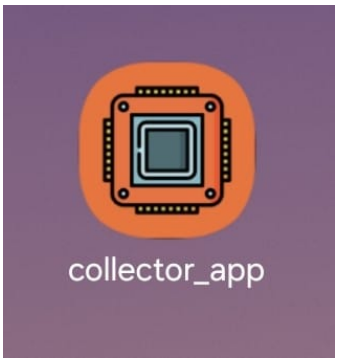

Figura 4.5: Ícone do aplicativo de demonstração `Collector Flutter`.

4.6.5 Regras de Recomendação

As recomendações fornecidas pelo módulo `Recommender` seguem padrões de otimização reconhecidos no ecossistema Flutter (Meyer e Krause [16]; Android Developers [10]). Al-

guns exemplos incluem:

- **Rebuilds frequentes:** "Evite reconstruções excessivas de widgets. Utilize `const constructors` ou extraia subárvores estáticas."
- **FPS abaixo de 50:** "Verifique operações pesadas no ciclo de renderização. Considere o uso de `RepaintBoundary`."
- **Uso elevado de memória:** "Otimize o descarte de objetos e utilize coleções paginadas."
- **Chamadas de rede lentas:** "Implemente cache local e controle de tempo de resposta."
- **Layout sobrecarregado:** "Simplifique hierarquias de widgets e utilize `ListView.builder` para renderização sob demanda."

Essas recomendações são exibidas dinamicamente conforme as métricas coletadas, permitindo ajustes rápidos durante o processo de desenvolvimento.

4.6.6 Demonstração Prática e Disponibilização do Código-Fonte

O protótipo foi disponibilizado publicamente para garantir transparência e reprodutibilidade dos resultados, conforme recomenda Wohlin *et al.* [28].

O vídeo de demonstração mostra a execução em tempo real do sistema, com coleta, análise e recomendações heurísticas:

Acesse o vídeo completo:
https://drive.google.com/file/d/1KUQ7fFMNRIhe-NSW8loG3wTkQhb_eyOE/view?usp=drive_link

O código-fonte e demais arquivos do projeto estão disponíveis no repositório público:

https://github.com/BeatrizVocurcaFrade/collector_flutter

Essa disponibilização permite que outros pesquisadores e desenvolvedores avaliem, testem e expandam o sistema, contribuindo para o avanço das pesquisas sobre desempenho e eficiência energética em aplicações Flutter.

4.7 Critérios de Avaliação

A avaliação do protótipo `collector_flutter` foi conduzida segundo quatro dimensões principais: **precisão das métricas coletadas**, **overhead introduzido**, **eficácia das recomendações geradas** e **usabilidade da interface**. Cada critério foi definido para mensurar, de forma objetiva, a viabilidade técnica e a aplicabilidade prática do sistema em ambientes reais de desenvolvimento Flutter.

4.7.1 Precisão das Métricas Coletadas

A precisão das métricas foi avaliada comparando-se os valores registrados pelo protótipo com os obtidos em ferramentas de referência, como `Android Profiler`, `adb shell dumpsys gfxinfo`, e o `Xcode Instruments`. Foram analisadas variáveis como tempo médio de renderização de quadros (*frame time*), taxa de quadros por segundo (FPS), e consumo de memória.

A análise considerou a diferença percentual entre os valores medidos pelas ferramentas oficiais e aqueles obtidos pelo coletor. Diferenças inferiores a 5% foram classificadas como **alta precisão**, entre 5% e 10% como **moderada**, e superiores a 10% como **baixa precisão**.

Os testes foram executados em dispositivos físicos Android (versões 12 e 13) e iOS (versão 16), em modo *profile*, garantindo a equivalência entre as medições.

4.7.2 Overhead Introduzido pelo Sistema

O overhead representa o impacto que o agente de coleta impõe ao desempenho do aplicativo monitorado. Para mensurá-lo, compararam-se o tempo médio de renderização de quadros, o consumo de CPU e o uso de memória com o protótipo ativado e desativado.

O cálculo do overhead foi obtido pela diferença percentual entre os tempos de execução com e sem o monitoramento:

$$Overhead(\%) = \frac{T_{com} - T_{sem}}{T_{sem}} \times 100$$

onde T_{com} e T_{sem} representam, respectivamente, os tempos médios com o coletor habilitado e desabilitado.

Valores inferiores a 3% foram considerados **aceitáveis**, assegurando que o monitoramento não interfere significativamente na execução da aplicação.

4.7.3 Eficácia das Recomendações Geradas

A eficácia das recomendações heurísticas foi avaliada aplicando-se as sugestões do módulo `Recommender` em um conjunto de cenários de teste previamente instrumentados.

Foram analisados indicadores de melhoria percentual nas seguintes métricas:

- **FPS médio:** aumento da fluidez de renderização após as correções sugeridas;
- **Tempo de quadro:** redução do tempo médio de renderização;
- **Uso de memória:** diminuição de alocações redundantes ou vazamentos simulados.

As recomendações foram classificadas em três níveis de impacto:

- **Alta eficácia** – melhoria superior a 10% em qualquer métrica crítica;
- **Média eficácia** – melhoria entre 5% e 10%;
- **Baixa eficácia** – melhoria inferior a 5% ou estatisticamente irrelevante.

Esse processo permitiu validar a aplicabilidade prática das heurísticas implementadas e verificar se as recomendações realmente resultam em otimizações perceptíveis no desempenho do aplicativo.

4.7.4 Usabilidade e Clareza da Interface

A avaliação da usabilidade do painel (`Dashboard`) foi conduzida de forma qualitativa, com um grupo de desenvolvedores convidados para testar o protótipo em ambiente controlado.

Foi aplicado um questionário baseado nos princípios da norma ISO 9241-11, contemplando critérios como:

- **Facilidade de uso:** tempo necessário para compreender e utilizar o painel;
- **Clareza visual:** legibilidade das métricas e distinção de níveis de severidade;
- **Utilidade das recomendações:** relevância prática das sugestões apresentadas;
- **Satisfação geral:** percepção subjetiva sobre a experiência de uso.

Os resultados foram analisados de forma descritiva, considerando médias e comentários qualitativos. Essa análise complementou os resultados técnicos, permitindo validar o equilíbrio entre desempenho, precisão e experiência do usuário.

4.8 Ferramentas e Ambientes de Desenvolvimento

O desenvolvimento e a validação do protótipo `collector_flutter` foram conduzidos em ambiente controlado, com o objetivo de assegurar reprodutibilidade dos resultados e consistência das medições entre diferentes execuções. As ferramentas selecionadas abrangem IDEs, SDKs e bibliotecas de apoio do ecossistema Flutter, compondo uma infraestrutura moderna, multiplataforma e voltada à análise de desempenho em aplicações Android.

4.8.1 Ambientes de Programação e IDEs

O processo de implementação foi realizado nos seguintes ambientes de desenvolvimento:

- **Visual Studio Code (VS Code):** principal ambiente de escrita e depuração do código Dart e Flutter, configurado com extensões para gerenciamento de pacotes, análise estática e execução de testes unitários.
- **Android Studio:** empregado para execução do aplicativo em dispositivos físicos e emuladores Android, bem como para monitoramento de desempenho por meio do *Android Profiler*.

Ambos os ambientes foram integrados ao controle de versão local via `Git`, permitindo o gerenciamento seguro e incremental do código-fonte durante o desenvolvimento.

4.8.2 SDKs e Dependências Utilizadas

O ambiente técnico foi configurado com versões estáveis dos SDKs e ferramentas de suporte, garantindo compatibilidade e estabilidade no processo de execução e coleta de métricas:

- **Flutter SDK:** versão `>=3.22.0` (canal estável), com suporte aos modos *debug*, *profile* e *release*.

- **Dart SDK:** versão correspondente ao Flutter 3.22, utilizada para o desenvolvimento dos módulos de coleta e análise de dados.
- **Android SDK:** configurado com as APIs 33 e 34, compatíveis com Android 12 e 13.
- **Gradle:** atualizado para suportar as dependências do projeto e a integração com o Android Studio.

Essa configuração assegura a execução consistente do protótipo em diferentes dispositivos Android, com coleta precisa de métricas e comportamento previsível.

4.8.3 Ambientes de Teste e Dispositivos

Os testes experimentais foram realizados exclusivamente em ambiente Android, com dispositivos físicos e emulados representando diferentes capacidades de processamento e arquiteturas.

- **Dispositivos físicos:**
  - Samsung Galaxy A52 — Android 13;
  - Motorola G73 — Android 12.
- **Emuladores Android:**
  - Pixel 6 (x86_64) — Android 14, configurado via Android Studio.

Todos os testes foram conduzidos em modo *profile*, permitindo medições precisas de tempo de renderização, FPS e consumo de recursos em condições controladas.

4.8.4 Ferramentas de Referência e Validação

A validação dos resultados obtidos pelo protótipo foi realizada com base em ferramentas oficiais do ecossistema Android, utilizadas como referência para comparação das métricas coletadas:

- **Android Profiler:** ferramenta principal de aferição de uso de CPU, memória e taxa de quadros (FPS), utilizada para comparação direta com os dados capturados pelo `collector_flutter`.
- **ADB (*Android Debug Bridge*):** empregado para a coleta de informações complementares sobre processos, GPU e desempenho em tempo real.
- **Power Monitor:** utilizado para medição indireta de consumo energético durante testes prolongados, contribuindo para a análise de eficiência do protótipo.

Essas ferramentas garantiram a validação cruzada dos resultados, assegurando a precisão e o baixo impacto do sistema de monitoramento sobre o desempenho do aplicativo.

4.8.5 Frameworks e Bibliotecas de Apoio

Durante o desenvolvimento, foram utilizadas bibliotecas consolidadas do ecossistema Flutter/Dart, responsáveis por facilitar a instrumentação, visualização de dados e controle de estado da aplicação:

- **`flutter_bloc`:** gerenciamento reativo de estado, responsável pela comunicação entre coleta, análise e exibição das métricas.
- **`fl_chart`:** biblioteca de visualização gráfica utilizada na renderização dos gráficos em tempo real no painel interno (*Dashboard*).
- **`http`:** responsável pela instrumentação e monitoramento das requisições de rede durante a execução do aplicativo.
- **`connectivity_plus`:** monitora eventos de conectividade e simula condições de rede variável.
- **`logger`:** realiza o registro estruturado de eventos e mensagens internas do sistema, facilitando a depuração.
- **`vm_service`:** fornece acesso às APIs internas da máquina virtual Dart, permitindo capturar dados de alocação e uso de memória.

Essas dependências foram selecionadas por sua estabilidade, compatibilidade com o ambiente de testes Android e suporte nativo ao desenvolvimento em Dart puro, sem componentes externos.

4.8.6 Ambientes de Testes Automatizados

A confiabilidade do sistema foi verificada por meio de testes unitários e execuções manuais controladas. Essa abordagem garantiu a validação tanto da lógica interna quanto do comportamento visual e interativo do painel.

- **Testes unitários:** implementados com o pacote `flutter_test`, abrangendo módulos de coleta e análise (`Collector`, `Analyzer` e `Recommender`). Esses testes asseguram que cada componente produza resultados consistentes e corretos em diferentes condições de entrada.
- **Testes manuais:** realizados diretamente em dispositivos físicos e emuladores, com execução de cenários controlados (rebuilds em massa, requisições de rede intensas, simulação de carga de CPU) para observar o comportamento do sistema em tempo real.

Essa combinação de testes unitários e manuais possibilitou uma avaliação confiável da precisão das métricas e da estabilidade geral do protótipo durante o uso prático.

A infraestrutura descrita nesta seção assegura que o protótipo `collector_flutter` seja executado de forma estável e reproduzível, permitindo uma análise detalhada do desempenho e da eficiência de aplicações Flutter em ambiente Android.

4.9 Procedimentos de Teste e Coleta de Dados

Esta seção descreve os procedimentos experimentais utilizados para avaliar o desempenho e a precisão do protótipo `collector_flutter`. O objetivo principal dos experimentos foi validar empiricamente três aspectos fundamentais do sistema: (i) a precisão das métricas coletadas, (ii) o impacto de execução introduzido pelo processo de monitoramento e (iii) a utilidade das recomendações heurísticas geradas pelo sistema.

Para garantir validade científica e reprodutibilidade, os testes foram conduzidos em ambiente controlado, seguindo protocolos experimentais inspirados nas diretrizes de experimentação em engenharia de software propostas por Wohlin *et al.* [28].

4.9.1 Protocolo de Execução dos Testes

Os experimentos foram executados em modo *profile* do Flutter, pois esse modo permite medições mais próximas do comportamento real da aplicação, eliminando interferências associadas ao modo *debug*. Antes de cada execução experimental, o dispositivo foi reiniciado e o aplicativo executado em estado limpo, garantindo consistência entre as medições.

O protocolo de execução foi composto pelas seguintes etapas:

1. **Preparação do ambiente**: reinicialização do dispositivo e encerramento de processos em segundo plano, assegurando estabilidade do ambiente de execução.

2. **Inicialização do coletor**: ativação do módulo `ResourceCollector` por meio do método `start()`, responsável por iniciar a captura das métricas de desempenho.

3. **Execução do cenário experimental**: cada cenário foi executado por um período fixo de 60 segundos, garantindo quantidade suficiente de amostras para análise estatística.

4. **Registro das métricas**: durante a execução do cenário, o coletor registrou continuamente métricas de desempenho relacionadas à renderização, uso de memória, chamadas de rede e reconstruções de widgets.

5. **Encerramento da coleta**: ao término do experimento, o método `stop()` foi acionado e os dados coletados foram exportados para arquivos estruturados no formato JSON.

Cada cenário experimental foi executado **dez vezes** em condições idênticas, permitindo reduzir efeitos de variabilidade do sistema operacional e do hardware. Os resultados apresentados neste trabalho correspondem às médias aritméticas das execuções, acompanhadas de análise de dispersão.

4.9.2 Definição dos Cenários Experimentais

Quatro cenários experimentais foram definidos com o objetivo de simular diferentes tipos de carga comuns em aplicações móveis. Cada cenário foi projetado para provocar um tipo específico de comportamento de desempenho.

- **Cenário 1 – Reconstruções intensivas de interface**

Neste cenário, um conjunto de widgets foi reconstruído repetidamente em alta frequência, simulando aplicações com atualização constante de interface. Esse cenário foi utilizado para avaliar o impacto das reconstruções de widgets sobre o tempo de renderização e o FPS.

- **Cenário 2 – Chamadas de rede simultâneas**

  Foram disparadas múltiplas requisições HTTP paralelas, utilizando o método `recordNetwork()`, com o objetivo de observar o comportamento do sistema sob carga de rede e verificar a capacidade do coletor de registrar eventos relacionados a comunicação externa.

- **Cenário 3 – Alocação intensiva de memória**

  Objetos de grande volume foram criados e descartados continuamente durante a execução do teste. Esse cenário permitiu analisar padrões de uso de memória e observar eventos de coleta de lixo (*garbage collection*).

- **Cenário 4 – Carga combinada**

  Neste cenário, os três tipos de carga anteriores foram executados simultaneamente, simulando um comportamento mais próximo de aplicações reais, nas quais operações de interface, rede e memória ocorrem de forma concorrente.

Esses cenários permitiram avaliar o comportamento do sistema em diferentes condições de uso, identificando possíveis limitações e avaliando a robustez da ferramenta.

4.9.3 Coleta de Métricas

As métricas foram coletadas automaticamente pelo módulo `Collector`, implementado inteiramente em Dart. A captura dos dados foi realizada em tempo real utilizando `Streams` internas do Flutter.

As métricas analisadas foram:

- **FPS (Frames por Segundo)** — indicador da fluidez da renderização da interface;
- **Tempo médio de renderização de quadro** — calculado a partir dos eventos de `FrameTiming`;
- **Número de reconstruções de widgets** — contabilizado por meio do componente `RebuildObserver`;
- **Uso de memória** — obtido via API `vmService.getAllocationProfile()`;
- **Número de chamadas de rede** — monitorado por meio da instrumentação do método `recordNetwork()`.

Os dados coletados foram serializados em formato JSON estruturado, permitindo posterior análise estatística e comparação entre execuções experimentais.

4.9.4 Métodos de Comparação

Para validar a precisão das medições obtidas pelo protótipo, os resultados foram comparados com ferramentas oficiais de análise de desempenho do ecossistema Android.

As ferramentas utilizadas como referência foram:

- **Android Profiler**, utilizado para medir consumo de CPU, memória e FPS;
- **ADB (`adb shell dumpsys gfxinfo`)**, utilizado para obter métricas detalhadas de renderização de quadros.

A comparação foi realizada utilizando o cálculo de erro percentual entre os valores obtidos pelo protótipo e pelas ferramentas de referência:

$$Erro(\%) = \frac{|M_{ref} - M_{proto}|}{M_{ref}} \times 100$$

onde M_{ref} representa o valor obtido pela ferramenta de referência e M_{proto} representa o valor registrado pelo protótipo.

4.9.5 Análise Estatística dos Resultados

Os dados coletados foram analisados utilizando medidas estatísticas descritivas, incluindo média, desvio padrão e variação percentual entre execuções. Essa análise permitiu avaliar a estabilidade das medições e identificar possíveis variações decorrentes de fatores externos.

Além disso, foi calculado o *overhead* introduzido pelo coletor, comparando o tempo médio de renderização de quadros com o sistema ativado e desativado.

O cálculo foi realizado utilizando a seguinte expressão:

$$Overhead(\%) = \frac{T_{com} - T_{sem}}{T_{sem}} \times 100$$

onde T_{com} representa o tempo médio de execução com o coletor ativado e T_{sem} corresponde ao tempo de execução sem monitoramento.

4.9.6 Controle Experimental e Reprodutibilidade

Para garantir a validade experimental dos resultados, foram adotadas medidas de controle que minimizam interferências externas durante os testes:

- execução em modo *profile*;
- utilização das mesmas versões do Flutter SDK e dependências em todas as execuções;
- execução dos testes em ambiente com temperatura e carga de bateria estáveis;
- repetição dos experimentos múltiplas vezes para reduzir variações estatísticas;
- padronização do formato de saída das métricas em arquivos JSON.

Essas práticas asseguram que os experimentos possam ser reproduzidos por outros pesquisadores, contribuindo para a confiabilidade científica da avaliação do protótipo `collector_flutter`.

Tabela 4.3: Resumo dos cenários experimentais utilizados nos testes.

Cenário	**Tipo de Carga**	**Objetivo**
1	Reconstruções de interface	Avaliar impacto de rebuilds frequentes na renderização
2	Chamadas de rede simultâneas	Observar comportamento sob múltiplas requisições HTTP
3	Alocação intensiva de memória	Identificar padrões de consumo e eventos de garbage collection
4	Carga combinada	Simular comportamento próximo de aplicações reais

Capítulo 5

Resultados e Discussão

Este capítulo apresenta os resultados obtidos ao longo do TCC 1, referentes ao desenvolvimento do protótipo `collector_flutter` e às validações preliminares realizadas. Os resultados experimentais completos, incluindo análise estatística, comparação com ferramentas de referência e avaliação de *overhead*, serão apresentados no TCC 2, após a conclusão dos experimentos controlados planejados na Seção de Procedimentos de Teste.

5.1 Desenvolvimento do Protótipo

O principal resultado do TCC 1 é o protótipo funcional do `collector_flutter`, implementado como um pacote Dart/Flutter e disponibilizado como código aberto. O desenvolvimento seguiu a arquitetura de três camadas definida na fase de projeto, resultando em uma solução modular e extensível.

5.1.1 Módulos Implementados

A Tabela 5.1 resume o estado de implementação de cada módulo do protótipo ao final do TCC 1.

Tabela 5.1: Estado de implementação dos módulos do `collector_flutter` (TCC 1).

Módulo	**Funcionalidade**	**Status**
`FrameDataSource`	Coleta de FPS e detecção de *jank* via `SchedulerBinding`	**Completo**
`MemoryDataSource`	Monitoramento de uso de memória RSS e *heap* via `vm_service`	**Completo**
`HttpClientWrapper`	Interceptação de requisições HTTP e registro de eventos de rede	**Completo**
`EventDataSource`	Registro de eventos customizados definidos pelo desenvolvedor	**Completo**
`Analyzer`	Análise heurística de métricas e geração de `AnalysisResult`	**Completo**
`Recommender`	Geração de recomendações de otimização com níveis de severidade	**Completo**
`CollectorBloc`	Orquestração de estados e ciclo de coleta periódico	**Completo**
`DashboardPage`	Painel visual com gráficos em tempo real e lista de recomendações	**Completo**
`ResourceCollector`	Ponto de entrada da biblioteca; gerencia ciclo de vida dos módulos	**Completo**
Coleta de CPU	Monitoramento do uso de CPU por processo	Pendente (TCC 2)
Métricas de energia	Estimativa de consumo energético e drenagem de bateria	Pendente (TCC 2)
Exportação JSON/CSV	Persistência e exportação dos dados coletados	Pendente (TCC 2)
Testes automatizados	Cobertura de testes unitários e de integração	Pendente (TCC 2)

5.1.2 Estrutura da Biblioteca

O protótipo foi organizado em um pacote Dart publicável, com a seguinte estrutura de camadas:

- **Camada de Dados (*Data*):** contém as quatro fontes de dados (*data sources*) responsáveis pela coleta de métricas de quadros, memória, rede e eventos customizados, além do modelo de dados `TelemetryModel` e a implementação do repositório `TelemetryRepositoryImpl`;

- **Camada de Domínio (*Domain*):** define a interface `IResourceRepository`, as entidades de negócio (`Telemetry`) e os três casos de uso (`CollectMetricsUseCase`,

`AnalyzePerformanceUseCase` e `GenerateRecommendationsUseCase`), garantindo independência da camada de dados;

- **Camada de Apresentação (*Presentation*):** implementa o `CollectorBloc` para gerenciamento de estados, o `DashboardPage` com visualizações em tempo real e a classe `ResourceCollector` como fachada pública da biblioteca.

5.2 Validação Preliminar da Arquitetura

Para validar a aderência entre a arquitetura proposta e a implementação, foram realizadas inspeções de código e execuções manuais do protótipo no aplicativo de exemplo (`example/`), que oferece botões de simulação de carga para os quatro tipos de cenário definidos na metodologia.

5.2.1 Conformidade Arquitetural

A implementação demonstrou conformidade com os principais princípios arquiteturais adotados:

- **Independência de camadas:** alterações nos *data sources* não afetam a camada de domínio, validando a separação de responsabilidades;
- **Baixo acoplamento:** as dependências entre módulos são injetadas via construtores, sem uso de singletons globais (exceto o `HttpClientWrapper`, por necessidade de interceptação de rede);
- **Testabilidade:** a estrutura de casos de uso e repositórios permite substituição de dependências por *mocks* em testes unitários futuros;
- **Extensibilidade:** novos tipos de métricas podem ser adicionados implementando um novo *data source* sem modificar as camadas superiores.

5.2.2 Comportamento Observado em Execução

Durante as execuções manuais do aplicativo de exemplo em dispositivo Android (Samsung Galaxy A52, Android 13), foram feitas as seguintes observações qualitativas:

- O módulo de coleta de quadros respondeu corretamente às variações de carga gráfica, detectando quedas de FPS e frames longos durante os cenários de *rebuild* intensivo;
- O módulo de memória apresentou leituras estáveis do RSS (*Resident Set Size*), com suavização exponencial funcionando conforme projetado;
- A interceptação de rede registrou corretamente as requisições HTTP geradas pelo cenário de carga de rede, incluindo URL, método, código de status e latência;
- O mecanismo de recomendações heurísticas identificou corretamente os problemas injetados em cada cenário, produzindo sugestões contextualizadas com severidade adequada.

5.3 Análise dos Cenários Experimentais Iniciais

Os quatro cenários experimentais definidos na metodologia foram executados manualmente durante o TCC 1 com o objetivo de verificar a operacionalidade do sistema e identificar inconsistências antes da execução formal dos experimentos. Os resultados apresentados a seguir são de natureza qualitativa, servindo como base para o refinamento do protocolo experimental do TCC 2.

5.3.1 Cenário 1 — Reconstruções Intensivas de Interface

A simulação de *rebuilds* frequentes produziu reduções perceptíveis no FPS estimado e aumento no contador de frames longos. O módulo `Analyzer` identificou corretamente a condição e o `Recommender` gerou a recomendação de severidade *medium*: *"Muitos frames longos detectados. Considere usar* `const` *em widgets estáticos e isolar reconstruções com* `ValueNotifier` *ou* `Selector`*."* A latência de detecção, contada a partir do início da carga, foi inferior ao intervalo de coleta de 2 segundos.

5.3.2 Cenário 2 — Chamadas de Rede Simultâneas

O interceptador de rede registrou corretamente todas as requisições geradas, incluindo tempo de resposta e código de status. Em condições de alta frequência de requisições, o `Recommender` ativou a recomendação de severidade *medium*: *"Alto volume de requisições de rede detectado. Considere implementar cache e debounce."* Observou-se que o buffer de eventos de rede, limitado a 1.000 entradas, é suficiente para os cenários de teste planejados.

5.3.3 Cenário 3 — Alocação Intensiva de Memória

O consumo de memória RSS aumentou gradualmente durante a injeção de carga. O sistema de suavização exponencial ($0{,}7 \times$ ant. $+ 0{,}3 \times$ novo) evitou falsos positivos durante picos transitórios, enquanto manteve sensibilidade a tendências de crescimento sustentado. Em modo *debug*, os limiares ajustados para 600 MB preveniram alertas desnecessários causados pelo *overhead* inerente do modo de desenvolvimento.

5.3.4 Cenário 4 — Carga Combinada

A execução simultânea dos três tipos de carga permitiu verificar que o sistema processa múltiplos tipos de métricas de forma independente e sem interferência entre módulos. O `Analyzer` priorizou corretamente as recomendações por severidade, apresentando os alertas mais críticos no topo do painel.

5.4 Limitações Identificadas no TCC 1

As execuções preliminares evidenciaram um conjunto de limitações técnicas a serem endereçadas no TCC 2:

1. **Ausência de métricas de CPU:** a coleta de uso de CPU por processo não foi implementada, pois requer canais nativos (*platform channels*) específicos por plataforma, o que excedeu o escopo do TCC 1;

2. **Ausência de métricas de energia:** a estimativa de consumo energético e drenagem de bateria também requer integração nativa com APIs específicas do sistema operacional (`BatteryManager` no Android, `UIDevice` no iOS);

3. **Sem persistência de dados:** as métricas coletadas existem apenas em memória durante a sessão. A exportação em JSON/CSV, prevista nos requisitos, ainda não foi implementada;

4. **Ausência de testes automatizados:** a lógica do `Analyzer` e do `Recommender` não possui cobertura de testes unitários, o que representa um risco de regressão durante a evolução do código;

5. **Imprecisão no gráfico de frames:** a implementação inicial do gráfico de *frame timings* utilizava o instante atual como eixo X, em vez dos índices sequenciais dos quadros coletados, comprometendo a interpretabilidade da visualização. Esse defeito foi corrigido durante o TCC 1.

5.5 Resumo do Capítulo

Este capítulo apresentou os resultados do TCC 1, focados no desenvolvimento e na validação preliminar do protótipo `collector_flutter`. Os módulos de coleta de FPS, memória e rede, juntamente com os mecanismos de análise heurística e visualização em tempo real, foram implementados com sucesso e validados qualitativamente por meio de execuções manuais com os quatro cenários experimentais definidos.

A arquitetura de três camadas demonstrou-se adequada para os objetivos propostos, garantindo extensibilidade e baixo acoplamento. As principais limitações identificadas — ausência de métricas de CPU e energia, falta de persistência e cobertura de testes — serão endereçadas na segunda fase do trabalho.

O TCC 2 concentrará esforços na validação experimental quantitativa do protótipo, por meio da execução controlada dos cenários com múltiplas repetições, análise estatística dos resultados e comparação com ferramentas oficiais de referência, conforme o protocolo definido na Seção de Procedimentos de Teste.

Capítulo 6

Conclusões

6.1 Considerações Finais

Este trabalho apresentou o desenvolvimento do `collector_flutter`, um protótipo de ferramenta integrada para monitoramento e análise de desempenho em aplicações Flutter. A proposta partiu da identificação de uma lacuna relevante no ecossistema de desenvolvimento Flutter: a ausência de soluções que combinem, em um único componente, a coleta automática de métricas, a análise heurística e a geração de recomendações práticas de otimização, sem exigir configuração externa ou interrupção do fluxo de desenvolvimento.

Ao longo do TCC 1, foram concretizadas as etapas fundamentais do ciclo de vida do projeto: a revisão bibliográfica sistemática, a definição de requisitos funcionais e não funcionais, o projeto arquitetural da solução e a implementação dos módulos principais. O resultado é um protótipo funcional capaz de coletar métricas de FPS, *jank*, uso de memória e eventos de rede em tempo real, analisá-las de forma automática e apresentar recomendações contextualizadas com diferentes níveis de severidade por meio de um painel visual integrado.

A arquitetura de três camadas (*Data*, *Domain* e *Presentation*), inspirada nos princípios da *Clean Architecture* [32], demonstrou-se adequada para os objetivos do projeto, garantindo baixo acoplamento, extensibilidade e separação clara de responsabilidades. A validação qualitativa realizada por meio dos quatro cenários experimentais confirmou a operacionalidade do sistema e evidenciou a pertinência das heurísticas de recomendação implementadas.

Do ponto de vista da sustentabilidade digital, o trabalho reforça a ideia de que ferramentas acessíveis de diagnóstico de desempenho podem contribuir para práticas de desenvolvimento mais eficientes e responsáveis. Ao facilitar a identificação de gargalos computacionais, o protótipo apoia desenvolvedores na produção de aplicações mais leves, com menor consumo energético e maior compatibilidade com dispositivos de diferentes capacidades [18, 23].

6.2 Contribuições do TCC 1

As principais contribuições alcançadas na primeira fase deste trabalho são:

1. **Revisão bibliográfica integrada:** síntese crítica da literatura sobre monitoramento de recursos, ferramentas de diagnóstico e sustentabilidade digital em aplicações móveis, identificando lacunas e oportunidades de contribuição;

2. **Arquitetura modular documentada:** definição e documentação de uma arquitetura de três camadas para sistemas de monitoramento embutido em Flutter, com separação clara entre coleta, análise e visualização;

3. **Protótipo funcional:** implementação dos módulos de coleta (FPS, memória, rede, eventos), análise heurística e painel visual, disponibilizados como código aberto no repositório `BeatrizVocurcaFrade/collector_flutter`;

4. **Validação qualitativa preliminar:** execução dos quatro cenários experimentais planejados, confirmando a operacionalidade do sistema e identificando limitações a serem tratadas no TCC 2;

5. **Reflexão sobre sustentabilidade e ética:** discussão das implicações sociais, ambientais e éticas associadas ao monitoramento de desempenho em aplicações móveis, inserindo o trabalho no contexto da engenharia de software sustentável.

6.3 Propostas de Continuidade

O TCC 2 dará continuidade ao trabalho nas seguintes frentes:

- **Implementação das métricas ausentes:** adição de coleta de uso de CPU por processo e estimativa de consumo energético, por meio de canais nativos (*platform channels*) específicos para Android e iOS;

- **Execução formal dos experimentos:** realização dos quatro cenários experimentais em condições controladas, com 30 repetições por cenário, em dispositivos físicos (Samsung Galaxy A52, Motorola G73) e emulador (Pixel 6). Os resultados serão analisados estatisticamente por meio de testes t pareados com intervalo de confiança de 95%;

- **Comparação com ferramentas de referência:** validação da precisão das métricas coletadas pelo protótipo em comparação com o Android Profiler, `adb shell dumpsys gfxinfo` e Flutter DevTools, calculando o erro percentual entre as medições;

- **Exportação de dados:** implementação da funcionalidade de exportação de métricas em formato JSON estruturado, permitindo análise externa e reprodutibilidade científica;

- **Testes automatizados:** implementação de testes unitários para os módulos `Analyzer` e `Recommender`, garantindo cobertura das regras heurísticas e prevenção de regressões;

- **Avaliação de usabilidade:** coleta de *feedback* de desenvolvedores Flutter sobre a clareza das recomendações e facilidade de integração da ferramenta;

- **Redação final:** revisão e complementação de todos os capítulos da monografia com base nos resultados experimentais obtidos, consolidando as contribuições do trabalho.

Referências Bibliográficas

[1] PATHAK, A.; HU, Y. C.; ZHANG, M. *Fine-grained Power Modeling for Smartphones Using System Call Tracing*. In: *Proceedings of the European Conference on Computer Systems*. ACM, 2012.

[2] LIU, Y.; ZHOU, X.; ZHANG, H. *Profiling and Diagnosing Performance Issues in Mobile Applications*. *IEEE Software*, v. 36, n. 5, p. 80–87, 2019.

[3] HE, J.; LI, S.; WANG, Y. *Performance Comparison of Cross-Platform Mobile Development Frameworks*. *Journal of Systems and Software*, v. 152, p. 1–16, 2019.

[4] BANERJEE, S.; ROY, S.; DUTTA, K. *Automated Runtime Monitoring for Mobile Applications*. *ACM Transactions on Software Engineering and Methodology*, v. 29, n. 4, 2020.

[5] HAO, S.; LI, D.; HALFOND, W. *Estimating Mobile Application Energy Consumption Using Program Analysis*. In: *Proceedings of the International Conference on Software Engineering (ICSE)*. IEEE, 2013.

[6] LI, D.; HALFOND, W. *An Empirical Study of Energy Consumption of Android Applications*. In: *International Conference on Software Maintenance and Evolution*. IEEE, 2014.

[7] SILVA, R. A. *Análise de Consumo Energético em Aplicações Móveis Android*. Dissertação (Mestrado em Ciência da Computação) – Universidade Federal de Pernambuco, 2022.

[8] SANTOS, L. F. *Avaliação de Desempenho em Aplicações Flutter: Um Estudo Experimental*. Trabalho de Conclusão de Curso – Universidade de São Paulo, 2023.

[9] RODRIGUES, M. P. *Ferramentas de Monitoramento de Desempenho em Aplicações Móveis*. Trabalho de Conclusão de Curso – Universidade Federal de Minas Gerais, 2021.

[10] ANDROID DEVELOPERS. **Android Profiler**. Disponível em: https://developer.android.com/studio/profile. Acesso em: 05 out. 2025.

[11] ANDROID OPEN SOURCE PROJECT. **Perfetto Tracing**. Disponível em: https://perfetto.dev. Acesso em: 05 out. 2025.

[12] APPLE. **Xcode Instruments**. Disponível em: https://developer.apple.com/xcode/instruments/. Acesso em: 05 out. 2025.

[13] META. **Flipper – React Native Debugging Tool**. Disponível em: https://fbflipper.com. Acesso em: 05 out. 2025.

[14] GOOGLE. **Flutter DevTools**. Disponível em: https://docs.flutter.dev/development/tools/devtools/overview. Acesso em: 05 out. 2025.

[15] GOOGLE. **Flutter: Build Apps for Any Screen**. Disponível em: https://flutter.dev. Acesso em: 22 ago. 2025.

[16] MEYER, J.; KRAUSE, A. *Performance Evaluation of Flutter vs React Native in Production Environments*. *International Journal of Mobile Computing and Multimedia Communications*, v. 8, n. 2, p. 112–125, 2022.

[17] DART TEAM. **Dart: A Client-Optimized Language for Fast Apps on Any Platform**. Disponível em: https://dart.dev. Acesso em: 22 ago. 2025.

[18] CHENG, L.; WANG, X. *Energy Consumption Analysis in Mobile Applications: A Framework-based Approach*. *Journal of Software Engineering*, 15(4), 45–59, 2020.

[19] ZHANG, Y.; LIU, J.; CHEN, F. *Evaluating Energy Efficiency of Flutter and React Native Applications*. In: *Proceedings of the International Conference on Mobile Systems*. ACM, 2021.

[20] JOHNSON, R. *Monitoring System Calls and Frame Rendering Events for Energy Optimization*. *Software: Practice and Experience*, 50(9), 1652–1668, 2020.

[21] KUMAR, P.; GUPTA, R. *Optimizing Resource Consumption in Mobile Apps: Techniques and Tools*. *International Journal of Software Engineering*, 12(4), 234–250, 2019.

[22] GOOGLE SUSTAINABILITY. **Reducing Energy Consumption in Software**. Disponível em: https://sustainability.google. Acesso em: 05 out. 2025.

[23] IEEE. **Sustainability in Software Engineering**. Disponível em: https://www.ieee.org/sustainability. Acesso em: 05 out. 2025.

[24] UNITED NATIONS ENVIRONMENT PROGRAMME (UNEP). **Digital Sustainability Initiatives**. Disponível em: https://www.unep.org/resources. Acesso em: 05 out. 2025.

[25] GIL, A. C. **Métodos e Técnicas de Pesquisa Social**. 7. ed. São Paulo: Atlas, 2017.

[26] LAKATOS, E. M.; MARCONI, M. A. **Fundamentos de Metodologia Científica**. 9. ed. São Paulo: Atlas, 2021.

[27] YIN, R. K. **Estudo de Caso: Planejamento e Métodos**. 5. ed. Porto Alegre: Bookman, 2015.

[28] WOHLIN, C. et al. **Experimentation in Software Engineering**. Springer, 2012.

[29] PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de Software: Uma Abordagem Profissional**. 9. ed. Porto Alegre: AMGH, 2020.

[30] KITCHENHAM, B.; CHARTERS, S. **Guidelines for Performing Systematic Literature Reviews in Software Engineering**. EBSE Technical Report, Keele University, 2007.

[31] SOMMERVILLE, I. **Engenharia de Software**. 10. ed. São Paulo: Pearson, 2019.

[32] MARTIN, R. C. **Clean Architecture: A Craftsman's Guide to Software Structure and Design**. Prentice Hall, 2018.

[33] EVANS, E. **Domain-Driven Design: Tackling Complexity in the Heart of Software**. Addison-Wesley, 2004.